From counterexamples to proofs

A tutorial on solver-aided reasoning in Haskell with SBV

LEVENT ERKÖK

Abstract

How can functional programmers apply automated reasoning techniques while keeping programs, specifications, and proofs in the same programming language and development environment? This tutorial develops a method of solver-aided reasoning in Haskell with SBV. The method rests on three decisions: how to formulate a question so that its answer means what we intend, how to represent programs and data so that a solver can reason about them, and how to structure an argument when automation alone is not enough. Counterexamples refine specifications, the same models support search, optimization, and synthesis, and induction and invariants extend proofs from simple properties to recursive programs. A suggested project carries the method into stateful verification. Recurring examples connect these ideas; a binary-search overflow found as a counterexample, for instance, later becomes part of a correctness proof. We close by examining what a successful check establishes, from generated code to the limits of the model. No background in automated reasoning is assumed, and an executable companion supplies checked examples and exercises.

Contents

1 Introduction 3

I. Formulating questions

2 Semantic domains and counterexamples 6
2.1 Symbolic values and their types 6
2.2 Symbolic operators 6
2.3 Validity, counterexamples, and vacuity 7
3 Independent specifications 8
3.1 Saturating arithmetic 8
3.2 A midpoint that leaves its interval 9
4 Bit-vectors and machine-word reasoning 10
4.1 Widths, signedness, and modular arithmetic 10
4.2 Masks, shifts, rotations, and bit fields 10
4.3 Widening, narrowing, and checked arithmetic 11
4.4 Bit tricks and independent specifications 11
5 Exact and floating-point arithmetic 12
5.1 Rationals and algebraic reals 12
5.2 Rounding modes and the limits of floating-point identities 13
6 Constraint solving and model enumeration 13
6.1 Satisfying models and finite enumeration 13
7 Optimization and competing objectives 14
7.1 Soft constraints, weights, and objective priorities 16

II. Choosing representations

8 Host evaluation and symbolic structure 16
8.1 Symbolic execution and path conditions 16
8.2 Haskell lists of symbolic values 17
8.3 Symbolic lists and recursive functions 17
8.4 Circuit equivalence through host-language recursion 18

9 Incremental solver interaction 19
9.1 Scoped assumptions and solver state 20
9.2 Unsatisfiable cores and explanations of failure 21
10 Abstraction through contracts 21
10.1 Generating symbolic types with `mkSymbolic` 22
10.2 Uninterpreted sorts and functions 23
10.3 Explicit assumptions and consistent contracts 24
11 Symbolic datatypes 24
11.1 Partial operations and exceptional inputs 25
11.2 Checking safety with `sAssert` and `safe` 27
12 Arrays and symbolic memory 27
12.1 Read-after-write laws and aliasing 28
12.2 Bounds and total memory models 28
13 Constraints over strings and sets 28
13.1 Regular expressions as specifications 28
13.2 Set membership and representation invariants 29
14 Quantifiers and program synthesis 30
14.1 Quantifier order and witness dependence 30
14.2 Expression search from examples 31
14.3 Counterexample-guided expression synthesis 31

III. Structuring arguments

15 Proofs by induction 33
15.1 A first inductive proof 33
15.2 Associativity of append 34
15.3 Solver obligations in a calculation 35
15.4 Generalization over accumulators 35
16 Kadane's algorithm and invariant discovery 36
16.1 Specification 36
16.2 Implementation 37
16.3 Invariant discovery 38
16.4 The proof 38
16.5 Lessons 40
17 Induction on decreasing measures 40
17.1 Expression trees 40
17.2 Binary search with a decreasing interval 41
18 McCarthy 91 and termination contracts 42
19 Failed TP proofs and counterexamples 43
19.1 Concrete inputs and observed values 44
19.2 A failed step in a true theorem 44
19.3 An incorrect justification 45
19.4 Function interpretations as counterexamples 46
20 Polymorphic and higher-order proofs 46
20.1 Polymorphic scripts, monomorphic proofs 46
20.2 The bookkeeping law 47
20.3 An arbitrary operation at a fixed type 48
20.4 Closures and higher-order functions 49
21 A verified tautology checker 49
21.1 Formulas, assignments, and evaluation 49
21.2 Normalization and a decreasing measure 50
21.3 Checking both branches consistently 51
21.4 Soundness through a generalized invariant 52

21.5 Completeness and a falsifying assignment 53

IV. Boundaries and context

22 C generation from verified functions 53
22.1 A verified function across the translation boundary 53
22.2 Executable validation and trust assumptions 54
23 The execution model and its proof obligations 54
23.1 Termination is a proof obligation 55
23.2 Finite lists and datatypes 55
23.3 Productivity and stream properties 56
24 Proof results, diagnostics, and trust 56
24.1 Failed checks and diagnostic steps 56
24.2 Encodings and solver selection 57
24.3 Model validation and proof dependencies 58
25 Connections to program reasoning 59
25.1 Testing and symbolic evaluation 59
25.2 Equational reasoning and proof guidance 59
25.3 Refinements, invariants, and the scope of a proof 60
26 Suggested projects 60
26.1 Project: from recursive invariants to state transitions 60
26.2 Project: mechanizing Bird's maximum-segment-sum derivation 61
27 Conclusion 62
Acknowledgments 62
References 62

1 Introduction

Haskell programmers often express the behavior of their code as properties. With QuickCheck (Claessen and Hughes 2000), these can be ordinary functions returning `Bool`; a test campaign searches for inputs that falsify them. SBV (Erkök 2026) lets the same programmer ask a stronger question. A property over *symbolic* values is translated to logic and handed to a solver for satisfiability modulo theories (SMT). Such a solver combines logical reasoning with theories such as integer arithmetic and fixed-width bit-vectors. When it can settle the query, it proves the property for every input or produces a concrete counterexample:

```
ghci> prove $ \x -> abs (x :: SInteger) .>= 0
Q.E.D.
ghci> prove $ \x -> abs (x :: SInt8) .>= 0
Falsifiable. Counterexample:
  s0 = -128 :: Int8
```

The two properties differ only in a type. Over mathematical integers, absolute values are nonnegative. Over signed bytes, 128 is not representable, so `abs (-128)` is −128. The solver did not test this value by luck: it solved a constraint whose domain contains all 256 inputs and reported a value that fails.

Our aim is to make this feedback part of ordinary development, with programs, specifications, and proofs expressed in the same programming language and development environment. The tasks extend beyond checking a property: constraints can describe a counterexample, a feasible solution, or a candidate program, and objectives can rank the solutions. A single solver call may settle an arithmetic property, while a recursive program may require a stronger statement, an induction hypothesis, or intermediate equalities. These activities share the three decisions developed throughout the tutorial: formulate a question whose answer has the intended meaning, choose a representation the solver can reason about, and supply an argument when automation alone is insufficient.

Functional programming provides useful structure for these decisions. Pure functions admit equational specifications; recursive definitions suggest induction principles; and higher-order combinators expose the algebraic laws on which a proof depends. SBV keeps programs, specifications, and proof scripts executable in Haskell while constructing solver queries from symbolic definitions. This reduces the need to translate definitions between tools and reconcile their syntactic and semantic differences by hand. It also lets counterexamples become inputs for ordinary evaluation, linking solver feedback back to the program. The underlying technique is *symbolic execution*: evaluation with symbolic inputs builds expressions describing many concrete executions at once (Section 8.1). The choice of symbolic domain determines what the resulting proof establishes.

The tutorial assumes familiarity with Haskell's algebraic datatypes, type classes, and monads, but no knowledge of SMT or theorem proving. It is intended for functional programmers and advanced students learning to mechanize program reasoning. The first three parts develop the three decisions; a fourth examines their boundaries and context:

- *Formulating questions.* Sections 2–7 begin with semantic domains and independent specifications. Mathematical integers, machine words, and floating-point arithmetic give different meanings to apparently similar properties. Saturating addition and a binary-search midpoint illustrate how counterexamples distinguish intended behavior from implementation details. Constraint solving and optimization turn specifications into search for feasible and preferred solutions, separating hard requirements from preferences.
- *Choosing representations.* Sections 8–14 distinguish Haskell evaluation from symbolic structure, with circuit equivalence as an example. Incremental queries organize repeated solver calls, and unsatisfiable cores help explain contradictory assumptions. Uninterpreted functions support abstraction through contracts; datatypes, arrays, strings, and sets provide different models of data. Quantifier order distinguishes finding one program that works for every input from finding an answer for each input. Expression search develops into synthesis by checking candidates and using counterexamples to refine the next search.
- *Structuring arguments.* Sections 15–21 combine equational calculations with induction. Generalization supplies the hypotheses needed for accumulator programs, including list reversal and Kadane's algorithm. Decreasing measures support proofs about expression trees and binary search; McCarthy 91 shows how termination can depend on a checked result contract. Failed TP obligations show how to distinguish a counterexample to a theorem from one to an intermediate step. Polymorphic and

higher-order proofs extend the method to reusable abstractions, and a verified tautology checker brings these techniques into a larger development.

- *Boundaries and context.* Sections 22–26 follow a verified function through C generation, then examine the execution model, termination obligations, and the scope of finite-data models. Diagnostics, solver selection, and proof dependencies help interpret successful and unsuccessful checks; connections to other approaches place the method in context. Suggested projects transfer invariant strengthening to stateful programs and explore the mechanization of Bird's algebraic derivation of maximum segment sum.

Recurring examples connect the parts: the midpoint counterexample returns in the binary-search proof, expression trees connect search and synthesis to induction, and accumulator generalization guides the suggested project on state transitions. Readers primarily interested in induction can proceed from Sections 2–3 and 8 to Section 15, returning to contracts, datatypes, and arrays as needed.

We verify functions expressed with SBV's symbolic operations; SBV does not translate arbitrary existing Haskell source into logic. In particular, a proof about mathematical integers or finite lists must not be silently transferred to machine integers, partial computations, or infinite lists. Making these choices explicit is part of the method, alongside writing and checking the proof.

Why not a proof assistant? Proof assistants such as Lean (de Moura and Ullrich 2021), Isabelle/HOL (Nipkow et al. 2002), Agda (Norell 2007), ACL2 (Kaufmann et al. 2000), Rocq (The Rocq Development Team nd), HOL4 (Slind and Norrish 2008), and PVS (Owre et al. 1992), among others, offer another valuable route. When a project can invest in expressing its programs and specifications in such a system and maintaining their correspondence with the Haskell implementation, that can be an excellent choice. Our goal here is to keep as much of this work as possible in the host language and its development environment. Programs, specifications, and proofs can then evolve together, without also maintaining a separate formalization in another language and reconciling its syntactic and semantic differences by hand. SBV still translates symbolic computations into solver logic, but the programmer works with Haskell definitions and proof scripts.

SMT automation is particularly helpful for obligations within the supported theories: the solver can discharge them without a user-written sequence of proof steps (Barbosa et al. 2022; de Moura and Bjørner 2008). Interactive proof development typically asks for more human guidance in choosing lemmas, organizing cases, and constructing proof scripts. This is a difference of emphasis: proof assistants also offer substantial automation, including interfaces to SMT solvers such as Isabelle's Sledgehammer (Blanchette 2026). AI agents are further changing the balance by selecting premises and generating proof steps, as demonstrated by LeanDojo (Yang et al. 2023). SBV's TP layer likewise needs user guidance when automation alone cannot settle a property; the aim is to express that guidance in Haskell and leave the individual obligations to the solver.

Following along. The standalone companion pins its dependencies, including an SBV 14.10 source snapshot, and declares the required language extensions. It was checked with GHC 9.14.1, Z3 5.3.2 (de Moura and Bjørner 2008), and CVC5 1.4.2 (Barbosa et al. 2022). Install

the solvers on PATH; Cabal does not install them. From the companion directory, build the examples and start GHCi:

```
cabal build tutorial-examples
cabal repl tutorial-examples
```

The session loads all imports, including Data.SBV.List qualified as SL, and enables the extensions used below. Run the checks with cabal run tutorial-examples; the README gives full validation instructions. The execution log records source hashes and exact tool versions. Each of the six numbered exercises names its checked solution in the companion; the suggested projects supply starting hints instead.

Listings are extracted from checked sources; transcripts and elisions are marked separately. Model values and their order can vary between solver versions, so validation checks their meaning rather than their printed form. Displayed transcripts use the current spelling Counterexample; the pinned snapshot prints Counter-example.

2 Semantic domains and counterexamples

2.1 Symbolic values and their types

SBV's central type is SBV a, a symbolic value whose concrete counterpart has type a. Abbreviations name the common cases: SBool, SInteger (unbounded integers), SWord8 through SWord64 and SInt8 through SInt64 (bit-vectors with wrap-around arithmetic), SReal (exact real numbers), SFloat and SDouble (IEEE-754), SString, and SList a. There are also arrays, sets, tuples, Maybe, Either, and user-defined algebraic datatypes.

These types are not interchangeable encodings of one numeric type; each is a different mathematical problem. The introduction showed that the choice between SInteger and SInt8 changes a theorem into a false statement. Floating point has its own surprises:

```
ghci> prove $ \(x :: SFloat) -> x .== x
Falsifiable. Counterexample:
  s0 = NaN :: Float
```

Equality in IEEE-754 is not reflexive, and the solver knows it. Choosing a symbolic type is choosing the semantics the solver will use, so it deserves the same care as choosing a type for a production program.

2.2 Symbolic operators

Arithmetic on symbolic values uses the ordinary Num class, so x + 1 and 2 * y work as expected. Comparisons cannot reuse Eq and Ord: their methods return a Haskell Bool, which must be known when Haskell evaluates it, while the result of comparing symbolic values may be unknown. SBV therefore provides .==, ./=, .<, .<=, .>, and .>=, which return SBool. Boolean connectives are .&&, .||, sNot, and implication .=>.

For the same reason, Haskell's if-then-else cannot branch on an SBool. The function ite :: Mergeable a => SBool -> a -> a -> a builds a symbolic choice that the solver resolves. This distinction between *Haskell control* and *symbolic control* recurs throughout the tutorial. Concrete arguments allow symbolic operations to compute immediately. For example, ite sTrue 10 20 :: SInteger produces the literal 10; no solver is involved.

Table 1. Entry points into SBV. All of them consume the same symbolic programs.

Interface	You supply	Successful result	Section
`prove`	A property	Validity or a counterexample	2–3
`sat`, `allSat`	Constraints on unknowns	One or all satisfying models	6
`optimize`	Constraints and objectives	Optimal models or objective bounds	7
`query`	A Haskell program driving the solver	Whatever the program computes	9
TP	Lemmas, induction, calculations	Checked proofs with dependencies	15–21
`compileToC`	A symbolic function	C source code	22

Solver calls can also be inconclusive. Enumeration can fail to terminate, and an optimization objective need not have an attained optimum.

2.3 Validity, counterexamples, and vacuity

Calling `prove` on a function of symbolic arguments creates a fresh unknown for each argument, evaluates the function to obtain a symbolic formula $P(\vec{x})$, and asks the solver whether $\neg P(\vec{x})$ is satisfiable. If it is not, P holds for every input and SBV prints `Q.E.D.` If it is, the satisfying assignment is a counterexample. A third outcome is possible: for undecidable or very hard problems, the solver may give up or time out. SBV reports this as an unknown result, which is neither a proof nor a refutation.

Unknowns can also be created by name inside the `Symbolic` monad, which gives readable counterexamples and allows assumptions:

```
prove $ do x <- sInteger "x"
           y <- sInteger "y"
           constrain $ x .> 0 .&& y .> 0
           pure $ x * y .> 0
```

Here `constrain` restricts attention to inputs that satisfy the assumption. A contradictory assumption makes every property provable, so assumptions should be kept as weak as the property permits. For assumptions A and conclusion P, `prove` searches for a model of $A \wedge \neg P$; it does not establish that A is feasible. For example,

```
ghci> prove $ \(x :: SWord8) -> x .> 255 .=> x .== 0
Q.E.D.
ghci> sat $ \(x :: SWord8) -> x .> 255
Unsatisfiable
```

The first answer is a vacuous proof: no byte satisfies its premise. Checking the premise with `sat` exposes the modeling mistake. Here and below, fresh arguments have the default universal interpretation under `prove` and existential interpretation under `sat`. Explicit mixed quantifiers are possible but outside this tutorial.

Table 1 lists the main ways of using a symbolic model. The rest of the tutorial visits each of them.

3 Independent specifications

A proof is only as good as the property it proves. The most useful properties compare an implementation with a specification written in a different, simpler way.

3.1 Saturating arithmetic

Consider addition on unsigned bytes that saturates at 255 instead of wrapping. An implementation can avoid overflow by testing before it adds, while the specification can use unbounded integers, where overflow cannot happen:

```
satAdd :: SWord8 -> SWord8 -> SWord8
satAdd x y = ite (x .> 255 - y) 255 (x + y)

satAddSpec :: SWord8 -> SWord8 -> SBool
satAddSpec x y = sFromIntegral (satAdd x y) .== ite (total .> 255) 255
    total
  where total = sFromIntegral x + sFromIntegral y :: SInteger
```

On concrete inputs, `satAdd 250 10` evaluates to 255 without a solver. The guard `x .> 255 - y` cannot overflow, because `255 - y` is representable for every byte `y`, and on the other branch `x + y` does not wrap. In the specification, `sFromIntegral` converts each byte to an integer *before* adding. Converting the already wrapped byte sum would repeat the implementation's arithmetic and could hide its mistakes.

```
ghci> prove satAddSpec
Q.E.D.
```

The proof covers all 65,536 input pairs. A tempting alternative implementation shows why the independent specification matters:

```
wrongAdd :: SWord8 -> SWord8 -> SWord8
wrongAdd x y = ite (x + y .> 255) 255 (x + y)

wrongAddSpec :: SWord8 -> SWord8 -> SBool
wrongAddSpec x y = sFromIntegral (wrongAdd x y) .== ite (total .> 255) 255
    total
  where total = sFromIntegral x + sFromIntegral y :: SInteger
```

```
ghci> prove $ do { x <- sWord8 "x"; y <- sWord8 "y"; pure (wrongAddSpec x y
    ) }
Falsifiable. Counterexample:
  x = 223 :: Word8
  y = 124 :: Word8
```

The sum 223 + 124 = 347 wraps to 91, and no byte is greater than 255, so the saturation branch is never taken. Had the specification also used byte arithmetic, it would have agreed with the bug. Your solver may report a different pair; any pair whose sum exceeds 255 is a valid counterexample, and the companion checks the reported pair with ordinary Haskell arithmetic.

Exercise 1. Write `satSub :: SWord8 -> SWord8 -> SWord8`, subtraction that saturates at 0, together with a specification over `SInteger`, and prove that they agree. Solutions: `satSub` and `satSubSpec`.

3.2 A midpoint that leaves its interval

Binary search supplies another example of why the arithmetic domain matters. The midpoint bug described by Bloch (2006) arises when `low + high` overflows before division by two. We can expose it without building an array: the local arithmetic already violates the intended bounds. The following functions, adapted from SBV's `BitPrecise.BrokenSearch` example, use signed 32-bit indices:

```
midPointBroken, midPointFixed :: SInt32 -> SInt32 -> SInt32
midPointBroken low high = (low +! high) /! 2
midPointFixed  low high = low +! ((high -! low) /! 2)
```

The operators `+!`, `-!`, and `/!`, from `Data.SBV.Tools.Overflow`, attach safety assertions to the arithmetic. The function `safe` checks these assertions under the supplied constraints and returns a result for each check:

```
midPointSafety :: (SInt32 -> SInt32 -> SInt32) -> IO [SafeResult]
midPointSafety f = safe $ do
  low  <- sInt32 "low"
  high <- sInt32 "high"
  constrain $ 0 .<= low .&& low .<= high
  output $ f low high
```

Running `midPointSafety midPointBroken` reports an addition overflow and a counterexample. For a simple instance, take $low = high = 2^{30}$. The sum wraps to -2^{31} and the computed midpoint is -2^{30}, outside the singleton interval. The companion checks both this instance and the solver's reported counterexample using ordinary Haskell arithmetic.

The repair computes the distance before adding half of it to the lower bound. Under $0 \le low \le high$, this avoids an overflowing intermediate sum. We can check all its arithmetic assertions:

```
ghci> all isSafe <$> midPointSafety midPointFixed
True
```

But an overflow-free expression could still compute the wrong answer. We need a separate specification. Signed 64-bit arithmetic can represent the sum of any two signed 32-bit inputs, so it gives a reference calculation:

```
midPointSpec :: (SInt32 -> SInt32 -> SInt32)
             -> SInt32 -> SInt32 -> SBool
midPointSpec f low high = 0 .<= low .&& low .<= high .=>
  sFromIntegral (f low high) .== (low64 + high64) `sDiv` 2
  where low64  = sFromIntegral low  :: SInt64
        high64 = sFromIntegral high :: SInt64
```

```
ghci> prove (midPointSpec midPointFixed)
Q.E.D.
```

As with saturating addition, both inputs are widened *before* the addition. These checks establish arithmetic safety and the value of the repaired midpoint; they do not yet show that binary search finds a target. We return to that algorithmic question in Section 17.2, after introducing arrays and induction.

4 Bit-vectors and machine-word reasoning

The midpoint bug showed why the arithmetic domain belongs in the specification. Machine words also carry representations: a byte may encode several fields, and an arithmetic identity may be used as a bit trick. This section moves from choosing the word's interpretation to proving that an encoding implements its intended operation. Each example has a small, independent specification.

4.1 Widths, signedness, and modular arithmetic

An n-bit word has 2^n bit patterns. `SWord8` interprets them as $0, \ldots, 255$; `SInt8` interprets them as $-128, \ldots, 127$. Addition and multiplication retain the low eight bits in either case, but comparison and division depend on signedness. Thus the pattern `0x80` denotes 128 as an unsigned byte and -128 as a signed byte. Ordinary arithmetic on these symbolic types wraps; the checked operators used in the midpoint example add separate overflow obligations. The underlying bit-vector theory makes this modular behavior precise (Barrett et al. 2025b). Division by zero has a separate, totalized meaning in SBV (Section 11.1).

Widths are part of the Haskell type. Besides the standard widths, `SWord 12` and `SInt 12` describe unsigned and signed twelve-bit words. A property checked at one width does not establish it at every width: a polymorphic helper can construct many instances, but each solver call fixes a width. We use bytes here so that the companion can also exhaust small input spaces independently.

The practical question is which behavior to specify. For a checksum, wraparound may be the intended operation. For an address or array bound, it may be a bug. Choosing `SInteger` merely to avoid overflow would change the latter verification problem; a faithful abstraction needs an explicit range condition or a proof relating the two representations.

4.2 Masks, shifts, rotations, and bit fields

Suppose a byte stores two four-bit fields. A mask selects the low field, a shift exposes the high field, and packing places them back in their positions:

```
lowNibble, highNibble :: SWord8 -> SWord8
lowNibble x = x .&. 15
highNibble x = x `shiftR` 4

packNibbles :: SWord8 -> SWord8 -> SWord8
packNibbles hi lo = (lowNibble hi `shiftL` 4) .|. lowNibble lo

fieldRoundTrip :: SWord8 -> SBool
fieldRoundTrip x = packNibbles (highNibble x) (lowNibble x) .== x
```

`prove fieldRoundTrip` establishes the round trip for every byte. The masks in `packNibbles` also define what happens when callers supply values larger than 15: their

high bits are discarded. If rejection is intended instead, that requires a different contract. The companion compares extraction with ordinary division and remainder on all 256 inputs.

The shift count in `shiftR x 4` is an ordinary Haskell `Int`. For a symbolic count, use operations such as `sShiftRight`; keep the count's domain explicit. Right shift of an unsigned word fills with zeros, whereas right shift of a signed word preserves its sign: shifting unsigned 128 right once gives 64, while shifting signed −128 gives −64. A rotation wraps discarded bits around instead of losing them. For example, the companion proves that rotating a byte left by three and then right by three restores it.

These operations manipulate bit patterns, while `.&&` combines symbolic Boolean propositions. Likewise `sTestBit x i` returns an `SBool`; Haskell's ordinary `testBit` cannot return a concrete `Bool` when `x` is unknown. This is another instance of the host/symbolic distinction we develop in Section 8.

4.3 Widening, narrowing, and checked arithmetic

Conversion is part of an algorithm, not a proof that a value fits. Narrowing an integer to a byte retains its residue modulo 256. Widening that byte back to an integer recovers the original value only within the representable range:

```
narrowExact :: SInteger -> SBool
narrowExact n = (0 .<= n .&& n .<= 255) .=>
  sFromIntegral (sFromIntegral n :: SWord8) .== n
```

`prove narrowExact` checks this representation invariant. The precondition is a requirement to establish at a conversion boundary. Assuming it in a caller would hide a possible truncation bug.

The position of a conversion matters too. Widening `x + y` after byte addition preserves its already-wrapped result. Widening `x` and `y` first performs the addition in the wider domain. For two unsigned bytes, `SWord16` suffices to hold their sum without overflow; an arbitrary longer chain of additions needs its own bound. Section 3's saturating adder uses this distinction to compare the machine implementation with an integer specification, while its midpoint example uses checked arithmetic to expose precisely where overflow can occur.

A useful workflow is to state the range invariant at the representation boundary, prove the arithmetic within that range, and check each narrowing operation. If overflow is intended, specify its modular effect instead of adding an unnecessary range restriction.

4.4 Bit tricks and independent specifications

A familiar test recognizes powers of two by clearing the lowest set bit. Rather than specifying this trick using another bit manipulation, enumerate the eight values it should accept:

```
powerOfTwo :: SWord8 -> SBool
powerOfTwo x = x ./= 0 .&& (x .&. (x - 1)) .== 0

powerOfTwoSpec :: SWord8 -> SBool
powerOfTwoSpec x = sOr [x .== literal (2 ^ k) | k <- [0 .. 7 :: Int]]

powerOfTwoCorrect :: SWord8 -> SBool
powerOfTwoCorrect x = powerOfTwo x .== powerOfTwoSpec x
```

`prove powerOfTwoCorrect` proves equivalence for all bytes. Subtracting one turns the lowest set bit to zero and every lower zero to one; intersecting with the original word removes that bit. The result is zero exactly when no other bit was set. The explicit nonzero test matters: zero also satisfies the masked equality, since its predecessor wraps to 255.

The finite specification describes unsigned byte powers of two, including 128. Changing the type to signed bytes while retaining a numerical "positive power of two" interpretation would require revisiting that last value. The companion also compares the implementation with Haskell's population count on all 256 bytes, independently of the symbolic enumeration.

This pattern scales to less transparent code: identify a simple specification, then ask whether the optimized bit-level implementation differs anywhere. `Documentation.SBV.Examples.BitPrecise.BitTricks` contains further examples; `PEXT_PDEP` studies extraction and deposition of selected bits. The lesson is to let the specification explain the operation rather than mirror the trick.

5 Exact and floating-point arithmetic

A choice between integers and machine words changes overflow behavior. Choosing between exact reals and floating point changes the arithmetic laws themselves. We now keep the expression nearly unchanged and observe what happens when its domain changes. This is useful both for numerical algorithms and for deciding what a real-valued model says about an eventual machine implementation.

5.1 Rationals and algebraic reals

`SReal` denotes exact mathematical reals, not a floating-point approximation. Rational constants and polynomial constraints can be expressed without rounding:

```
exactThird :: SReal -> SBool
exactThird x = x / 3 + x / 3 + x / 3 .== x

positiveSquareRoot :: IO SatResult
positiveSquareRoot = sat $ do
  x <- sReal "root"
  pure $ x .> 0 .&& x * x .== 2
```

The first property is valid for every real input. The second query asks for the positive square root of two. Its result is an algebraic number, represented by SBV as an `AlgReal`; the printed decimal prefix is only a display of that value. The companion rechecks the returned model against the defining equations without converting it to `Double`.

Exact rational arithmetic can also be modeled with `SRational`, whose values are ratios of integers. The constructor `n .% d` forms such a value from a numerator and denominator; for example, three copies of `1 .% 3` sum exactly to one. Use a nonzero denominator. A rational domain excludes the square root of two, whereas `SReal` includes it. These domains are therefore not interchangeable merely because both represent one third exactly. Division by zero is assigned a value in both symbolic domains; Section 11.1 explains the convention and how to state a nonzero-divisor contract. Symbolic rational encodings also introduce integer arithmetic on their components, so exactness alone says nothing about solver difficulty.

Nor does `SReal` make arbitrary real analysis automatic. The polynomial constraint above is very different from a specification involving a general transcendental function. Establish

which operations the chosen theory models, and retain the distinction between an exact real proof and a floating-point implementation. The extension-field bounds in Section 7 address yet another question: whether an optimization bound is attained.

5.2 *Rounding modes and the limits of floating-point identities*

`SFloat` and `SDouble` model binary floating-point values and operations, including signed zeros, infinities, and NaN. SBV exposes the rounding mode through operations such as `fpAdd`; ordinary `(+)` uses round-to-nearest, ties-to-even. SMT-LIB's floating-point theory specifies these operations separately from real arithmetic (Tinelli and Brain 2024).

At 2^{24}, consecutive binary32 numbers are two units apart. Adding one lies halfway between them: ties-to-even rounds down here, while rounding toward positive infinity rounds up. We can also state associativity while requiring all inputs and intermediate results to be finite:

```
roundingExample :: SBool
roundingExample =
  let x = 16777216 :: SFloat
  in fpAdd sRNE x 1 .== x .&&
     fpAdd sRTP x 1 .== 16777218

finiteAssociativity :: SFloat -> SFloat -> SFloat -> SBool
finiteAssociativity x y z =
  let left  = (x + y) + z
      right = x + (y + z)
  in sAnd (map fpIsPoint [x, y, z, x+y, y+z, left, right])
     .=> left .== right
```

The companion proves `roundingExample` and refutes `finiteAssociativity` at the fixed instance $x = 2^{24}$, $y = 1$, and $z = -2^{24}$, which satisfies the finiteness restrictions:

$$(x + y) + z = 0, \qquad x + (y + z) = 1.$$

In the left expression, the first addition loses the unit. In the right, $1 - 2^{24}$ is exactly representable, so the final addition recovers it. No NaN, infinity, or overflow is needed. This is why a real-valued algebraic simplification can invalidate a floating-point algorithm.

Exceptional values still deserve explicit treatment. `fpIsPoint` excludes NaN and infinities but includes signed zeros and subnormal values. IEEE equality makes NaN unequal to itself and equates positive and negative zero. If the property concerns floating-point objects rather than numerical equality, inspect `fpIsEqualObject` and state the intended distinction. An error-bound proof needs assumptions about inputs and the rounding of each operation; replacing all operands by `SReal` silently removes those obligations. The library's `Examples.Misc.Floating` module offers further counterexamples. Section 11.1 contrasts IEEE division by zero with the conventions for integers, reals, and rationals.

6 Constraint solving and model enumeration

6.1 *Satisfying models and finite enumeration*

Proving a property asks whether every input satisfies it. The dual question asks whether *some* input satisfies a constraint, and which one. The function `sat` returns one satisfying model, and `allSat` enumerates all of them. Which bytes are unchanged by squaring?

```
idempotentBytes :: IO AllSatResult
idempotentBytes = allSat $ \(x :: SWord8) -> x * x .== x
```

```
ghci> idempotentBytes
Solution #1:
  s0 = 1 :: Word8
Solution #2:
  s0 = 0 :: Word8
Found 2 different solutions.
```

The answer is a small theorem in disguise: $x(x-1) \equiv 0 \pmod{256}$, and since x and $x-1$ are coprime and $256 = 2^8$, one of them must be divisible by 256. `allSat` repeatedly finds a model and excludes it before searching again. The finite byte domain makes a complete enumeration possible here; on an unbounded domain, asking for all models may never finish.

7 Optimization and competing objectives

A satisfying model witnesses feasibility; an optimization result also describes how a feasible choice compares with alternatives. This adds a modeling decision: which quantities define a better answer, and how should competing objectives be ordered? The function `optimize` ranks feasible solutions by named objective *measures*: `minimize` a cost or `maximize` a profit. SBV's `Metric` class supports symbolic integers, reals, and bit-vectors, among others. These objectives differ from the decreasing measures used later to establish termination. A workshop makes two products with profits 2 and 3, subject to two resource limits:

```
production :: IO OptimizeResult
production = optimize Lexicographic $ do
  x <- sInteger "x"
  y <- sInteger "y"
  constrain $ x .>= 0 .&& y .>= 0
  constrain $ x + 2 * y .<= 10
  constrain $ 3 * x + y .<= 15
  maximize "profit" (2 * x + 3 * y)
```

Running `production` gives $x = 4$, $y = 3$, and profit 17. Here `constrain` sets hard requirements. Over `SInteger` this is integer linear programming; using `SReal` instead allows fractional production quantities. The resource bounds ensure that the maximum is attained. In general, check for infeasibility, unbounded objectives, and unattained bounds: maximizing a real x subject to $x < 1$ has supremum 1 but no optimal model.

Combining measures. The first argument of `optimize` selects an `OptimizeStyle`:

- `Lexicographic` optimizes objectives in declaration order. Once the first objective is optimal, the second breaks ties, and so on. For example, maximize profit first, then minimize the number of units produced without sacrificing profit.
- `Independent` uses Z3's *box* mode: optimize each objective separately under the same constraints, returning a model for each. These individual optima need not be jointly attainable.
- `Pareto Nothing` enumerates nondominated objective vectors: improving one objective requires worsening another. Their set is the *Pareto front*. Use `Pareto (Just`

`n`) to request at most n such points; an infinite front makes unrestricted enumeration nonterminating.

For comparison, maximize nonnegative integers x and y with $x + y \leq 10$. Lexicographic order x then y gives $(10, 0)$; box gives separate maxima $x = 10$ and $y = 10$; the Pareto front consists of the eleven pairs with $x + y = 10$. Box does not promise the infeasible pair $(10, 10)$.

Bounds in an extension field. Extension-field results apply to both `SInteger` and `SReal`: unbounded objectives can yield `oo` or `-oo`. Real objectives can additionally require infinitesimals. For example, independently minimize and maximize a positive real:

```
realBounds :: IO OptimizeResult
realBounds = optimize Independent $ do
  x <- sReal "x"
  constrain $ x .> 0
  minimize "lower" x
  maximize "upper" x
```

Both independent results report these extension-field bounds in the tested configuration:

```
lower = epsilon :: Real
upper =      oo :: Real
```

Here `epsilon` denotes a positive infinitesimal: the infimum is 0 but is unattained. It is not a small floating-point tolerance. The symbol `oo` denotes positive infinity: the maximum is unbounded. Neither is a concrete real assignment to `x`. SBV exposes such bounds through `getModelObjectives` as `GeneralizedCV` values, using `ExtendedCV` for extension-field expressions. Changing `sReal` to `sInteger` in this example makes the minimum 1, while the maximum remains `oo`.

Strict-bound pitfall. Infinitesimal results are not reliable for every formulation or Z3 version: issue 5314 reports this minimization returning 1 without an unknown result (Erkök and Bjørner 2021). Check strict-bound behavior; replacing $>$ by $\geq$ changes the problem.

Solver support and arithmetic limits. Z3 is currently the only optimization backend supported by SBV. SMT-LIB has not standardized optimization commands; SBV uses Z3's extensions for objectives and priorities (Microsoft ndb). Optimization is therefore less portable across solvers than ordinary satisfiability checking.

Products of unknowns, such as `x * y`, introduce nonlinear arithmetic, which can be difficult even without optimization; over integers, satisfiability is undecidable (Microsoft nda). Z3's polynomial real procedures include optimization support since 5.3, but do not provide a general solution for nonlinear integer or mixed problems (Microsoft ndb). Prefer faithful linear formulations and distinguish feasible candidates from established optima.

Exercise 2. With `production`'s constraints, maximize `x + 2*y`, then minimize `x + y` lexicographically. Compare with the opposite priority order. Solutions: `productionLex` for the original order and `productionLexReversed` for the reverse.

7.1 Soft constraints, weights, and objective priorities

A soft constraint is a preference that the solver may violate at a cost. Hard constraints still determine feasibility. Suppose a choice must be 0, 1, or 2, and two competing preferences favor opposite ends of this range:

```
softPreferences :: IO OptimizeResult
softPreferences = optimize Lexicographic $ do
  x <- sInteger "choice"
  constrain $ 0 .<= x .&& x .<= 2
  assertWithPenalty "prefer-zero" (x .== 0) (Penalty 2 Nothing)
  assertWithPenalty "prefer-two"  (x .== 2) (Penalty 3 Nothing)
```

`Penalty 2 Nothing` charges two units when its Boolean assertion is false; `Nothing` places it in the default objective group. The two penalties in this example add. Choosing 0 costs 3, choosing 1 costs 5, and choosing 2 costs 2, so the optimum is 2. The companion checks that answer against all feasible choices, not merely against the objective value reported by the solver.

Weights express tradeoffs within a group. A sufficiently large collection of lower-weight preferences can outweigh a higher-weight one. This differs from lexicographic objectives, where no improvement in a later objective compensates for worsening an earlier one. Penalties can also be assigned named groups, which become separate objectives; choose the optimization style and objective order deliberately when combining groups with explicit measures.

Keep a requirement hard if no amount of benefit elsewhere should justify breaking it. Conversely, encoding every preference with `constrain` can make an otherwise useful planning problem infeasible. A satisfying model, a minimum total penalty, and a lexicographically preferred model answer different questions, even when their Haskell predicates look similar.

8 Host evaluation and symbolic structure

An SBV interaction separates two kinds of work: Haskell evaluates the program to build a symbolic formula, and the solver reasons about that formula. Query programs can alternate between these activities. Knowing which side handles which part of a program explains how recursion behaves.

8.1 Symbolic execution and path conditions

Ordinary execution computes with particular inputs. *Symbolic execution* instead represents inputs by symbols and computes expressions in those symbols. At a branch, the guard determines which inputs follow each alternative. The conjunction of conditions along a path is its *path condition*; solving it can produce a concrete input that follows that path. King's foundational account (King 1976) develops this connection between symbolic execution and program testing. Baldoni et al. (2018) survey subsequent techniques, including ways to explore and merge paths.

In SBV, Haskell evaluation constructs these expressions through operations on symbolic values. Recall `satAdd x y = ite (x .> 255 - y) 255 (x + y)`. With unknown byte inputs, the comparison produces a symbolic guard c. Under assumptions A, the two alternatives have path conditions $A \wedge c$ and $A \wedge \neg c$: the first returns 255, the second the byte sum. An

ordinary Haskell `if` requires a concrete `Bool` to choose its branch. SBV's `ite` instead combines both outcomes into a conditional expression, representing their effects in one formula. It does not require a separate solver query for each path.

For a desired property Q of the inputs and result, a counterexample query has the form

$$A(x, y) \ \wedge \ \neg Q\big(x, y, \mathrm{ite}(c, 255, x + y)\big).$$

Here arithmetic retains its byte semantics. A satisfying assignment supplies inputs that violate Q and determines which branch they take. Unsatisfiability establishes Q for every modeled input satisfying A. Symbolic execution builds the expression; SMT solving answers the resulting logical question. The `satAddSpec` proof uses this process with an independently widened sum.

SBV implements this approach as a Haskell library: the programmer supplies definitions over symbolic types and explicit symbolic choices. It does not automatically translate arbitrary Haskell source. Merging branches and sharing expression nodes (Section 23) avoid duplicating some work, but do not remove the growth in formulas or the difficulty of solving them. Recursion adds a further choice: unfold a known amount of Haskell structure, or represent a recursive function in the logic and supply induction when needed. The next two subsections distinguish these cases.

8.2 Haskell lists of symbolic values

A Haskell list `[SInteger]` has a spine that Haskell owns and elements that may be symbolic. Functions such as `sum`, `map`, and `foldr` work on it unchanged, because they only rearrange the spine and apply `Num` operations to the elements:

```
nonNegSum :: Int -> Predicate
nonNegSum n = do
  xs <- mapM (\i -> sInteger ("x" ++ show i)) [1 .. n]
  pure $ sAll (.>= 0) xs .=> sum xs .>= 0
```

For example, `prove (nonNegSum 5)` establishes the property for five symbolic integers. Haskell unfolds `sum` into `x1 + x2 + x3 + x4 + x5` before the solver sees anything; the recursion has disappeared. This style is ideal for fixed-size problems: circuits, unrolled loops, puzzles on fixed boards. The price is that the theorem is about lists of length five, not all lists.

8.3 Symbolic lists and recursive functions

The type `SList Integer` denotes a *finite* sequence whose length and contents may both be unknown. There is no fixed length bound, but infinite lists and partial values are outside this domain. Throughout the tutorial, a theorem about "every list" means every finite list of values in the modeled element type (Section 23.2). To state such a property, we need a function whose recursion is part of the logic, not something Haskell unfolds. Writing a naive recursive Haskell function over `SList` does not work: whether the list is empty is unknown, so Haskell would unfold both branches forever. SBV's `smtFunction` solves this problem. It gives the function a name, and recursive calls become calls to that named function in the logic:

```
sumL :: SList Integer -> SInteger
sumL = smtFunction "sumL" $ \xs -> [sCase| xs of
          []     -> 0
          y : ys -> y + sumL ys
       |]
```

The `sCase` quasi-quoter provides pattern matching on symbolic values. The definition reads like ordinary Haskell, but SBV translates it to a single recursive SMT-LIB definition (Barrett et al. 2025a). Because `sumL` is a function of symbolic values, it still computes on literals: `sumL [1,2,3]` evaluates to `6` without a solver. When a recursive definition is registered for symbolic reasoning, SBV checks termination; for `sumL`, it automatically finds the list length as a decreasing, nonnegative measure. Termination makes the definition admissible; it does not prove arbitrary properties of it.

Now try the obvious generalization of `nonNegSum`:

```
z3Timeout5 :: SMTConfig
z3Timeout5 = z3 { solverSetOptions = solverSetOptions z3 ++ [SetTimeOut
    5000] }

sumWithoutInduction :: IO ThmResult
sumWithoutInduction =
  proveWith z3Timeout5 $
    \xs -> SL.all (.>= 0) xs .=> sumL xs .>= 0
```

The property is true, but ordinary SMT reasoning about recursive definitions does not reliably establish it. The configuration preserves Z3's default options and limits each solver call to five seconds; an unknown answer is expected with the tested configuration. It says nothing about the truth of the property. Section 15 supplies an induction argument that works for lists of every finite length.

The general rule is: *recursion over Haskell structure happens while building the problem; recursion through `smtFunction` becomes part of the problem.* The former fixes the amount of unfolding before solving; the latter allows statements about structures of unbounded size. Neither representation guarantees that the resulting solver query will be easy.

8.4 Circuit equivalence through host-language recursion

A circuit gives a concrete use for recursion over host-language structure. The following ripple-carry adder consumes a Haskell list of bit pairs, from least to most significant. Each recursive call constructs one full-adder stage; the carry is symbolic, but the number of stages is known:

```
import qualified Documentation.SBV.Examples.BitPrecise.Adders as Adders
```

```
ripple :: SBool -> [(SBool, SBool)] -> ([SBool], SBool)
ripple carry [] = ([], carry)
ripple carry ((a,b):rest) = (sumBit : bits, finalCarry)
  where sumBit = a .<+> b .<+> carry
        next = (a .&& b) .|| (carry .&& (a .<+> b))
        (bits, finalCarry) = ripple next rest

rippleByte :: SWord8 -> SWord8 -> SWord8
```

```
rippleByte x y = fromBitsLE bits
  where (bits, _) = ripple sFalse (zip (blastLE x) (blastLE y))

circuitEquivalence :: SWord8 -> SWord8 -> SBool
circuitEquivalence x y =
  rippleByte x y .== x + y .&&
  rippleByte x y .== fromBitsLE lookaheadBits
  where (lookaheadBits, _) = Adders.lookaheadAdd sFalse (zip (blastLE x) (
    blastLE y))
```

`blastLE` exposes the eight bits of each byte as a fixed Haskell list, and `fromBitsLE` reconstructs the result. Dropping the final carry implements addition modulo 256. A specification of the full nine-bit result would instead retain that carry.

The library's `lookaheadAdd` constructs carries from the generate and propagate signals of lower positions. Unlike the ripple construction, its carry expressions do not follow a sequential chain of intermediate carries. `prove circuitEquivalence` checks both the ripple adder against native byte addition and its agreement with the lookahead circuit. It establishes functional equivalence, not a physical timing or gate-area claim.

Haskell has already traversed the eight positions before the solver sees the formula. There is no list induction in this proof: it is a finite circuit question. The same generator can build a different fixed width, but that gives a different theorem. For a proof covering all widths together, see `Documentation.SBV.Examples.TP.Adder`; the induction techniques later in this tutorial explain why that is a different task.

9 Incremental solver interaction

The functions `prove`, `sat`, and `optimize` manage solver interactions on the caller's behalf. Many applications need a conversation: ask, inspect the answer, and ask a refined question. SBV's `query` mode hands control of a live solver session to a Haskell program. The following function finds the smallest integer greater than 10 that leaves remainder 3 when divided by 7 and remainder 4 when divided by 5:

```
smallest :: IO Integer
smallest = runSMT $ do
  x <- sInteger "x"
  constrain $ x .> 10
  constrain $ x `sMod` 7 .== 3
  constrain $ x `sMod` 5 .== 4
  query $ let descend best = do
                r <- checkSat
                case r of
                  Sat   -> do v <- getValue x
                              constrain $ x .< literal v
                              descend (Just v)
                  Unsat -> maybe (io $ fail "no solution") pure best
                  _     -> io $ fail "inconclusive"
          in descend Nothing
```

```
ghci> smallest
24
```

The constraints before `query` set up the problem as usual. Inside the query, `checkSat` asks for a model, and `getValue` reads the value of `x` in that model. That value is an ordinary `Integer`, so Haskell can branch on it and use it to build the next constraint, `x .< literal v`, which asks for a smaller solution. When no smaller solution exists, the last model found is the minimum. The `case` on the solver's answer is Haskell control; the constraint that it adds is symbolic. Keeping the two apart is what makes the solver usable as a component of a larger algorithm.

Three properties of this loop matter. First, successful candidates are strictly decreasing *integers* bounded below by 11. After the first candidate v, there can be at most $v - 11$ further decreases. Thus the loop terminates provided each solver call returns. The existence of a minimum alone would not justify this reasoning over reals: the candidates could decrease forever without reaching it. Second, the solver may return the candidates in any order. Our loop does not depend on that order; it only relies on unsatisfiability at the end. Third, the loop handles an unknown answer separately: an inconclusive solver response is not evidence that no smaller value exists.

Queries can do much more: `push` and `pop` add and retract temporary assumptions, named constraints allow unsatisfiable cores to be extracted, and new unknowns can be created in the middle of a session with `freshVar`. The last point is less trivial than it looks: a new variable may have a type, such as a user-defined datatype, that the solver has not seen yet. SBV tracks the declarations each new term needs and sends them before the term is used, so queries compose without regard to the order in which types are first used.

Exercise 3. Change `smallest` to find the *largest* solution below 100. (The answer is 94.) Solution: `largestBelow100`.

9.1 Scoped assumptions and solver state

Incremental constraints persist until they are explicitly retracted. A scope allows a speculative branch to add facts without changing the surrounding problem:

```
scopedQuery :: IO [CheckSatResult]
scopedQuery = runSMT $ do
  x <- sInteger "x"
  constrain $ x .>= 0
  query $ do
    push 1
    constrain $ x .< 0
    impossible <- checkSat
    pop 1
    temporary <- checkSatAssuming [x .== 3]
    restored <- checkSatAssuming [x .== 4]
    pure [impossible, temporary, restored]
```

The result is `[Unsat, Sat, Sat]`. Within the pushed scope, `x < 0` conflicts with the permanent nonnegative bound. `pop 1` removes that constraint. Each `checkSatAssuming` then supplies an assumption for just that check: asking for 3 does not prevent the following

check from asking for 4. This is useful for exploring alternatives against one shared background model.

A model belongs to the check that produced it. Call `getValue` only after a satisfiable answer, and read needed values before changing constraints or popping their scope. A value already returned to Haskell is concrete data; it does not become an additional solver constraint unless reintroduced with `literal`. This distinction is what lets `smallest` remember its previous candidate while excluding that candidate from later checks.

Balance pushes and pops along every control-flow path. A forgotten pop can turn a later query into an unintentionally stronger problem. Treat `unknown` as a separate outcome rather than backtracking as if a branch had been proved impossible. The synthesis loop in Section 14 uses these same query operations to accumulate new examples.

9.2 Unsatisfiable cores and explanations of failure

An unsatisfiable answer says there is no model, but does not by itself explain which requirements conflict. Name the constraints and request an unsat core:

```
coreExample :: IO [String]
coreExample = runSMT $ do
  setOption $ ProduceUnsatCores True
  x <- sInteger "x"
  namedConstraint "nonnegative" $ x .>= 0
  namedConstraint "negative"    $ x .< 0
  namedConstraint "upper"       $ x .< 100
  query $ do
    answer <- checkSat
    case answer of
      Unsat -> getUnsatCore
      _     -> io $ fail "expected contradictory bounds"
```

In the checked run the core contains `nonnegative` and `negative`. Those two bounds already contradict each other; `upper` adds nothing to that explanation. The returned names refer to assertions in this solver session, not to Haskell source locations or to a diagnosis of the programmer's intent.

A core is a sufficient conflicting subset, not necessarily the smallest one or even a minimal one. Do not make application logic depend on a particular order or on the exclusion of redundant constraints. The companion accepts a returned subset of the known names and independently rechecks that subset for unsatisfiability. Unnamed background assertions remain part of the context in which a core is sufficient.

This turns the earlier vacuity warning into a diagnostic technique. Before trusting a theorem under elaborate assumptions, check those assumptions for satisfiability. If they conflict, a core can isolate the requirements to inspect. The repair is a modeling decision: weakening whichever constraint is easiest to remove may change the intended problem.

10 Abstraction through contracts

Sometimes the implementation of an operation is unavailable, expensive to encode, or intentionally hidden behind an interface. We can reason about its observable contract instead.

The modeling decision is which properties of the operation are available to clients, and which conclusions remain conditional on those properties.

10.1 Generating symbolic types with `mkSymbolic`

So far we have used types for which SBV already provides symbolic support. For a user-defined type, `mkSymbolic` generates that support from its Haskell declaration. It uses Template Haskell: GHC inspects the declaration at compile time and inserts generated declarations into the module. Consider an expression language with integer leaves and two kinds of binary node:

```
data Expr = Num Integer
          | Add Expr Expr
          | Mul Expr Expr
          deriving Show
mkSymbolic [''Expr]
```

This single declaration gives us payload fields, several constructors, and recursive children. It will serve as a running example for search, synthesis, and induction. With the `TemplateHaskell` extension enabled, the two quotes in `''Expr` denote a Template Haskell *type name*, not a string or a value of type `Expr`. The list supplies the names of the types to process. This top-level call is a declaration splice, written in the form that permits omitting the explicit `$(...)` wrapper. Place it after the datatype declaration and before code using the generated names. No solver is called at this stage.

The generated alias is `SExpr = SBV Expr`, together with instances that let SBV represent symbolic expressions and decode solver models as ordinary `Expr` values. The generated interface lets us build and inspect expressions whose leaves or entire subtrees are unknown:

Generated interface	Role
`SExpr = SBV Expr`	Symbolic type alias.
`sNum :: SInteger -> SExpr`	Construct a symbolic numeric leaf.
`sAdd :: SExpr -> SExpr -> SExpr`	Construct an addition node.
`isNum :: SExpr -> SBool`	Test a constructor symbolically.
`getMul_1 :: SExpr -> SExpr`	Select the first field of a multiplication node.

For example, `sAdd (sNum n) subtree` has a known outer constructor while `n :: SInteger` and `subtree :: SExpr` can both be symbolic. An arbitrary `e :: SExpr` may also have an unknown outer constructor; `isNum e` is therefore a symbolic proposition. Field accessors require the corresponding constructor condition: `getMul_1 e` is meaningful as the left child only when `isMul e` holds. Section 11 uses this condition in a search.

Generated case-analysis support lets `sCase` select a branch when the constructor is unknown. To see it in use, count binary nodes in an expression, then state that the count is nonnegative:

```
exprSize :: SExpr -> SInteger
exprSize = smtFunction "exprSize" $ \e -> [sCase| e of
  Num _   -> 0
  Add a b -> 1 + exprSize a + exprSize b
  Mul a b -> 1 + exprSize a + exprSize b
  |]

exprSizePos :: TP (Proof (Forall "e" Expr -> SBool))
exprSizePos = inductiveLemma "exprSizePos"
  (\(Forall e) -> exprSize e .>= 0) []
```

We write the recursive function ourselves; `mkSymbolic` supplies the symbolic representation and case-analysis support it uses. The numeric leaf has size zero, and either binary node adds one to the sizes of both children.

`mkSymbolic` also generates the *structural induction principle* for datatypes with constructors. For `Expr`, proving a property of every `Num` leaf and proving it for `Add` and `Mul` assuming it for both children establishes it for every finite expression. TP's `inductiveLemma` uses this principle in `exprSizePos`. Its `Forall` binder states the property for every expression, and the empty list supplies no additional lemmas. For `Num`, the obligation is $0 \geq 0$. For either binary constructor, the hypotheses give nonnegative child sizes, making their sum plus one nonnegative. The generated principle connects these three cases to the universal theorem. Running `runTP exprSizePos` checks the proof; Section 15 develops the TP interface in detail. The size function and its theorem return as the measure for the mirror proof in Section 17.

What is generated depends on the declaration. For an enumeration, induction reduces to checking each constructor. A constructor-free declaration instead introduces a nonempty opaque sort, with no constructor structure and no induction principle; we use one next. For recursive datatypes such as `Expr`, values are finite trees of unbounded size (Section 23.2). The generated schema supplies the induction rule, while the programmer supplies the property and any guidance needed to prove its cases. It does not translate arbitrary Haskell functions or automatically prove their properties.

10.2 Uninterpreted sorts and functions

To hide a representation completely, give `mkSymbolic` a constructor-free declaration:

```
data Key
mkSymbolic [''Key]
```

This generates `SKey = SBV Key` for a nonempty uninterpreted sort, despite there being no ordinary Haskell constructors with which to build a key. An uninterpreted function supplies an operation on that domain with a type but no defining equations:

```
keyCode :: SKey -> SInteger
keyCode = uninterpret "keyCode"

keyCongruence :: SKey -> SKey -> SBool
keyCongruence x y = x .== y .=> keyCode x .== keyCode y
```

We have specified neither a representation of keys nor a fixed number of distinct keys.

`prove keyCongruence` succeeds because functions respect equality: equal keys have equal codes. The converse does not follow. The operation may map every key to the same integer unless an additional contract rules that out. Likewise, naming two symbolic keys does not assert that they differ; use an explicit disequality when the model needs distinct objects.

This is useful abstraction when a client's theorem depends only on a few laws. The solver is free to choose any interpretation compatible with those laws, so a theorem must hold for all such interpretations. A counterexample can therefore expose a missing interface condition even when the intended concrete implementation would never exhibit the reported behavior. Section 19 shows how to read function interpretations in counterexamples.

10.3 Explicit assumptions and consistent contracts

Consider an integer operation that promises strict progress. Its implementation is hidden, but its contract suffices to prove progress after two calls:

```
advance :: SInteger -> SInteger
advance = uninterpret "advance"

advanceContract :: (SInteger -> SInteger) -> SBool
advanceContract f = quantifiedBool $ \(Forall n) -> f n .> n

twoAdvances :: SInteger -> SBool
twoAdvances n = advanceContract advance .=> advance (advance n) .>= n + 2
```

`quantifiedBool` turns the quantified property into an `SBool` that can be used as a premise. The solver instantiates it at `n` and at `advance n`. Because the domain is integers, each strict increase is at least one; hence the total increase is at least two. The same premise over reals would not justify that numerical bound.

The theorem is conditional. To use it for a concrete implementation, establish that implementation's contract as a separate obligation. The function `(+1)` supplies a simple consistency witness, and the companion proves it satisfies `advanceContract`. Adding the requirement `advance 0 .== 0` would contradict the contract at zero. Under both premises every conclusion would be vacuously true; the companion checks this contradiction as well.

For a larger collection of contracts, use a satisfiability check and named constraints as above. Quantified satisfiability can itself be inconclusive; a concrete implementation satisfying the contracts can provide a constructive consistency argument. Keep these explicit premises distinct from TP's `axiom`: an axiom is a trusted dependency, whose use is recorded in the proof's trust information (Section 24.3). The bookkeeping proof later uses an arbitrary operation in just this modular way, with its algebraic laws as the theorem's premises.

11 Symbolic datatypes

Return to the `Expr` datatype introduced in Section 10.1. Its constructors define both candidates for a search and structures over which a theorem may quantify. We now supply an evaluator and search for an expression with a specified value; later we prove that a transformation preserves evaluation (Section 17):

```
eval :: SExpr -> SInteger
eval = smtFunction "eval" $ \e -> [sCase| e of
          Num n   -> n
```

```
          Add a b -> eval a + eval b
          Mul a b -> eval a * eval b
       |]

puzzle :: IO SatResult
puzzle = sat $ \e -> eval e .== 42 .&& isMul e .&& sNot (isNum (getMul_1 e)
    )
```

The evaluator is an `smtFunction`, so it is a recursive definition in the logic, and `sCase` matches on an expression whose constructor may be unknown. The puzzle asks the solver to *synthesize* an expression: a multiplication whose left argument is not a number, evaluating to 42.

```
ghci> puzzle
Satisfiable. Model:
  s0 = Mul (Mul (Num 1) (Num (-1))) (Num (-42)) :: Expr
```

The model is an ordinary Haskell value of type `Expr`, which we can inspect, print, or evaluate with a separate Haskell interpreter. The companion does exactly that to check the answer independently of the solver. Different solver versions may find different expressions.

The constructor test `isMul e` matters: an accessor such as `getMul_1 e` describes the intended field only when `e` has that constructor. Also, this is synthesis from a structural constraint and one evaluation constraint, not a proof that the expression implements a function on every input. The search has no explicit depth bound and is not guaranteed to terminate for arbitrary constraints.

The same datatype can be the subject of proofs: a property of `eval` over all expressions concerns finite expression trees of unbounded size, and may need induction. Infinite or partial trees are outside the model (Section 23.2). Section 17 returns to `Expr` with a proof that a recursive transformation preserves evaluation.

11.1 Partial operations and exceptional inputs

A symbolic operation is a total function in the logic, even at inputs where its ordinary Haskell counterpart would fail. Such an input does not necessarily raise an exception or generate a safety obligation. Instead, the operation has either a specified result or an unspecified value. These cases must be understood before transferring a symbolic theorem to an executable program:

Operation and exceptional input	Symbolic behavior
`sDiv` or `sQuot` by zero on integers or machine words	Quotient zero; `sMod` and `sRem` return the dividend.
`(/)` by zero on `SReal` or `SRational`	Result zero, including zero divided by zero.
Division on `SFloat` or `SDouble`	IEEE behavior: e.g., $1/(+0)$ is positive infinity and $0/(+0)$ is NaN.
`SM.fromJust SM.sNothing`, empty-list head, or a field selector for the wrong constructor	An unspecified value of the result type.

The division conventions are choices made by SBV, not claims about Haskell's partial integer operations or mathematical division. For example, `prove integerZeroDivision` checks that zero divisors give quotient zero and remainder equal to the dividend. It does not establish that a program avoids zero divisors. The companion checks the table's numeric cases, including signed and unsigned bytes. Check the particular API: the explicitly Euclidean `sEDiv` and `sEMod` leave their zero-divisor results unconstrained.

Unspecified does not mean an exception. Here `SM` is `Data.SBV.Maybe`, whose symbolic `fromJust` is a field selector. Compare a query about its invalid input with a guarded law:

```
unspecifiedPayload :: Integer -> IO SatResult
unspecifiedPayload n = sat $
  (SM.fromJust SM.sNothing :: SInteger) .== literal n

guardedPayload :: SMaybe Integer -> SBool
guardedPayload m = SM.isJust m .=> SM.sJust (SM.fromJust m) .== m
```

Both `unspecifiedPayload 0` and `unspecifiedPayload 1` return `Satisfiable`. In each separate query, the solver can choose an interpretation of the selector that meets the constraint. Neither result says that ordinary Haskell `fromJust Nothing` can safely return an integer. Nor is this a fresh random value on each use: within one interpretation the same selector on the same input has the same value. The companion checks that requiring this single term to equal both zero and one is unsatisfiable. The `guardedPayload` theorem uses `isJust` to establish when extracting and rebuilding the payload recovers the original value.

Making the contract explicit. If zero divisors are forbidden, state a precondition such as `y ./= 0` and establish it at each call site. Proving a theorem under that assumption does not prove that callers satisfy it. If failure is an expected result, expose it in the type instead:

```
headOption :: SList Integer -> SMaybe Integer
headOption xs = ite (SL.null xs) SM.sNothing (SM.sJust (SL.head xs))

divOption :: SInteger -> SInteger -> SMaybe Integer
divOption x y = ite (y .== 0) SM.sNothing (SM.sJust (x `sDiv` y))

headOptionSpec :: SList Integer -> SBool
headOptionSpec xs = SM.isJust (headOption xs) .== sNot (SL.null xs)
```

The wrappers return `Nothing` for an empty list or zero divisor and `Just` on valid inputs. Their contracts therefore avoid depending on the exceptional-input values in the table. Alternatively, `SM.fromMaybe d m` provides an explicit default: it returns `d` for `Nothing` and the payload for `Just`. The companion checks both constructors. Choose between a precondition, an optional result, and a default according to the intended API. The binary-search contract in Section 17.2, for example, establishes `isJust` as well as the bounds and value of its returned index.

Finally, symbolic totality does not protect the Haskell code that constructs the query. `ite` builds a symbolic choice; it cannot generally make an ordinary Haskell exception in a branch safe merely because the solver will later rule that branch out. Our wrappers use symbolic selectors and arithmetic, not host-language attempts to destruct an unknown value. Treat host

exceptions, specified symbolic defaults, and unspecified symbolic values as three different behaviors.

11.2 Checking safety with `sAssert` and `safe`

`sAssert` attaches a required condition to a symbolic computation; `safe` checks whether such assertions can fail and reports counterexamples. Assertions can express nonzero divisors, valid indices, or constructor preconditions. For example, we can require a payload before extracting it:

```
checkedFromJust :: SMaybe Integer -> SInteger
checkedFromJust m =
  sAssert Nothing "fromJust requires Just" (SM.isJust m) (SM.fromJust m)
```

The first argument omits optional call-stack information; the message identifies the assertion. The condition must hold for the returned value to meet our contract. Unlike `constrain`, which assumes a condition, `sAssert` creates an obligation to check:

```
ghci> safe checkedFromJust
[fromJust requires Just: Violated. Model:
  s0 = Nothing :: Maybe Integer]
```

With an explicit precondition, the same assertion succeeds:

```
safe $ do
  m <- free "m" :: Symbolic (SMaybe Integer)
  constrain $ SM.isJust m
  output $ sAssert Nothing "fromJust requires Just"
             (SM.isJust m) (SM.fromJust m)
```

```
[fromJust requires Just: No violations detected]
```

Here `constrain` supplies the precondition, `sAssert` checks the requirement, and `output` registers the result for checking. Safety is established under the precondition; callers must still satisfy it.

Checked arithmetic operations in `Data.SBV.Tools.Overflow` insert assertions for overflow and underflow. Ordinary arithmetic and selectors do not acquire checks automatically: `safe` does not flag `SM.fromJust SM.sNothing` by itself. An empty list of safety results means no assertions were checked; it is not evidence that all exceptional inputs have been ruled out.

12 Arrays and symbolic memory

The expression trees above describe data by constructors. Memory is often more naturally described by what can be observed at an address. SMT arrays support this second representation: proofs use read-after-write laws and equality of mappings, rather than induction over a sequence of stored cells. An `SArray Integer Integer` is a total mapping from integer addresses to integer values. Use `sArray "mem"` to create an unknown array and `readArray mem i` to read a symbolic address. Updates are functional: `writeArray mem i v` returns an updated array, preserving `mem` as the previous state.

12.1 Read-after-write laws and aliasing

The array theory supplies read-after-write laws: the updated address returns the new value; all other addresses retain their values. Equality is *extensional*: equal arrays agree at every address, and conversely (Tinelli 2024). Thus writes to distinct addresses commute:

```
writesCommute :: SArray Integer Integer
              -> SInteger -> SInteger -> SInteger -> SInteger -> SBool
writesCommute a i j x y =
  i ./= j .=>
    writeArray (writeArray a i x) j y .==
    writeArray (writeArray a j y) i x
```

```
ghci> prove writesCommute
Q.E.D.
```

The proof covers every initial array, both addresses, and both values. The solver establishes equality of entire arrays directly, without traversal or induction.

Omit `i ./= j` and the property is false: the addresses may *alias*. For example, with $i = j = 0$, $x = 1$, and $y = 2$, the left side stores 2 at address 0 while the right side stores 1. The last write wins, regardless of the initial contents. The companion obtains a counterexample from the solver and checks its updates independently using Haskell association-list lookup.

12.2 Bounds and total memory models

SMT arrays have no built-in length or out-of-bounds error. For a buffer of length `n`, prove `0 .<= i .&& i .< n` at each access; assuming the bound would exclude unsafe accesses. `constArray v` initializes every address to `v`, while `lambdaArray` defines values by index, for example to update an interval and preserve `readArray mem i` elsewhere. The index type determines the domain: `Word8` has 256 addresses and `Integer` infinitely many. These total mappings differ from the finite recursive structure of lists and datatypes (Section 23.2).

Exercise 4. Prove that writing twice to the same address equals performing only the last write, for every array, address, and pair of values. Solution: `lastWriteWins`.

13 Constraints over strings and sets

Representations can expose a specification directly. A string validator can be compared with a regular language, while an access-control invariant can be stated as set inclusion. These examples use solver theories to express the question at the level of the intended data, without first encoding characters or membership tables as integer arithmetic.

13.1 Regular expressions as specifications

Suppose a protocol accepts identifiers consisting of exactly three lowercase ASCII letters. One implementation checks the length and each character; an independent specification uses a regular expression:

```
import qualified Data.SBV.RegExp as RE
```

```
identifierPattern :: RE.RegExp
identifierPattern = RE.Loop 3 3 (RE.Range 'a' 'z')

validIdentifier :: SString -> SBool
validIdentifier s = SL.length s .== 3 .&&
  sAnd [lower (s SL.!! literal i) | i <- [0 .. 2 :: Integer]]
  where lower c = literal 'a' .<= c .&& c .<= literal 'z'

identifierSpec :: SString -> SBool
identifierSpec s = validIdentifier s .== RE.match s identifierPattern
```

`RE.Loop 3 3` means exactly three repetitions. Matching covers the whole string, not a substring. `prove identifierSpec` checks both directions: every accepted string belongs to the language, and every string in the language is accepted. A one-way implication would leave either rejection of valid identifiers or acceptance of invalid ones unexplained.

String operations are available through `Data.SBV.List`. Here Haskell enumerates three positions in a symbolic string. The length check puts these positions within bounds whenever validation succeeds. The companion compares the validator with a separate Haskell predicate on ordinary strings, including empty strings, uppercase letters, digits, and non-ASCII characters.

Regular languages give useful specifications for lexical formats. SMT-LIB provides string and regular-expression operations (Tinelli et al. 2025), but a pattern is not a complete parser for an application protocol. This example intentionally chooses ASCII letters; it asserts nothing about case folding, Unicode normalization, or byte encodings. Make such transformations explicit when a production validator depends on them. The library's `Examples.Strings.RegexCrossword` and `SQLInjection` demonstrate larger combinations of symbolic strings and constraints.

13.2 Set membership and representation invariants

Suppose `active` contains enabled users and `allowed` contains those permitted by policy. Their representation invariant is inclusion. Enabling one user preserves it if that user is allowed:

```
import qualified Data.SBV.Set as SS
```

```
activate :: SInteger -> SSet Integer -> SSet Integer
activate = SS.insert

activationPreserves :: SInteger -> SSet Integer -> SSet Integer -> SBool
activationPreserves user active allowed =
  active `SS.isSubsetOf` allowed .&& user `SS.member` allowed
  .=> activate user active `SS.isSubsetOf` allowed
```

The theorem covers every pair of sets and every integer user ID. A membership argument explains it: a member of the updated set was either already active or is the newly inserted user. In either case the premises place it in `allowed`. Unlike a list representation, a set has no meaningful element order or duplicates; equality compares membership.

`SSet Integer` can denote infinite sets, including the universal set and complements of finite sets. Do not assume it can be enumerated or converted to a finite Haskell list. A model may describe a set by finitely many exceptions to a default membership value, much like the function models later in the paper. This is different from the finite recursive structure of an ADT. If the domain is `Word8`, there are only 256 possible members, but the choice of finite domain is still part of the specification.

Sets are a good fit when the observable operations are membership, union, intersection, difference, and inclusion. A queue or ordered collection needs a representation that preserves more information. Choosing a set because it simplifies the solver query would be unsound if ordering matters to the client.

14 Quantifiers and program synthesis

So far, counterexamples have helped repair a supplied implementation. They can also guide the construction of one. We will synthesize the coefficients of an expression template, verify the resulting candidate on every integer input, and feed any counterexample back into the search. The expression language from Section 11 and the query machinery from Section 9 now serve one combined purpose.

14.1 Quantifier order and witness dependence

Quantifier order states which choices may depend on which inputs. Compare one offset that works for every integer with an offset chosen after seeing the input:

```
oneOffset :: IO SatResult
oneOffset = sat $ \(Exists offset :: Exists "offset" Integer)
                    (Forall x :: Forall "x" Integer) -> offset .== x + 1

offsetPerInput :: IO SatResult
offsetPerInput = sat $ \(Forall x :: Forall "x" Integer)
                         (Exists offset :: Exists "offset" Integer) ->
    offset .== x + 1
```

The first query is unsatisfiable: the same offset cannot equal both 1 at input 0 and 2 at input 1. The second is satisfiable, with the input-dependent choice $x+1$. `Exists` and `Forall` specify logical quantifiers; the Haskell type annotations give their names and sorts. They are different from Haskell's `forall`, which quantifies type variables.

Program synthesis usually requires $\exists p.\, \forall x.\, \mathit{correct}(p, x)$: choose one program that works on every input. Reversing the quantifiers may allow a different program for each input and therefore solve a weaker problem. The synthesis parameters need not determine a constant output; they determine a program that subsequently receives `x`. Quantifier order controls the dependence of the *choice of program*, not whether the program can read its argument.

The companion checks both offset queries. General quantified formulas are harder than these examples, and a satisfying quantified formula need not provide a convenient executable witness. The following loop separates candidate search and verification into simpler calls and returns ordinary Haskell coefficients.

14.2 Expression search from examples

Reuse `Expr` to build a family of affine expressions. A candidate has the shape $ax + b$; the input is supplied as a numeric leaf when the expression is built. We search for coefficients between -3 and 3:

```
-- The input is inserted as a numeric leaf in the existing Expr language.
affineAt :: SInteger -> SInteger -> SInteger -> SInteger
affineAt a b x = eval (sAdd (sMul (sNum a) (sNum x)) (sNum b))

targetAt :: SInteger -> SInteger
targetAt x = x + x + x + 1

affineParameters :: Symbolic (SInteger, SInteger)
affineParameters = do
  a <- sInteger "a"
  b <- sInteger "b"
  constrain $ sAnd [(-3) .<= a, a .<= 3, (-3) .<= b, b .<= 3]
  pure (a,b)
```

This is template synthesis: the tree shape is fixed by Haskell, while two constants are unknown. It is not an unrestricted search over expression trees. There are 49 possible coefficient pairs, but each denotes a function on all integers. `targetAt` supplies an independent expression for the desired behavior rather than reusing `affineAt`.

First require agreement on a finite collection of examples:

```
exampleSearch :: [Integer] -> IO SatResult
exampleSearch inputs = satWith z3Timeout5 $ do
  (a,b) <- affineParameters
  pure $ sAnd [affineAt a b (literal x) .== targetAt (literal x) | x <-
    inputs]
```

`exampleSearch [0]` fixes $b = 1$ and leaves a unconstrained within its range. For instance, $a = -3, b = 1$ fits that example but fails at $x = 1$. Adding examples improves the specification of the candidate; it is not in general a universal correctness proof. The companion re-evaluates the returned coefficients using the ordinary Haskell interpreter `evalC`.

A grammar is a modeling choice. If it excludes every correct implementation, an unsatisfiable search means only that no candidate in this family fits the examples. Conversely, extending the grammar can make search much harder. We keep it small so that the feedback loop, rather than expression-generation machinery, is the main idea.

14.3 Counterexample-guided expression synthesis

Once the coefficients are concrete, verification asks for an input on which the candidate disagrees with the specification:

```
candidateCounterexample :: (Integer, Integer) -> IO (Maybe Integer)
candidateCounterexample (a,b) = runSMTWith z3Timeout5 $ do
  x <- sInteger "input"
  constrain $ affineAt (literal a) (literal b) x ./= targetAt x
  query $ do
    result <- checkSat
    case result of
      Sat   -> Just <$> getValue x
      Unsat -> pure Nothing
      _     -> io $ fail "candidate verification was inconclusive"
```

A satisfiable answer returns a new example. An unsatisfiable answer establishes agreement on every integer, since the query searched for an arbitrary violating input. An unknown answer establishes neither, so the helper reports it as inconclusive. The coefficients are fixed during this check: allowing them to change would ask the wrong question.

The candidate solver retains all examples accumulated so far:

```
synthesizeAffine :: IO (Integer, Integer)
synthesizeAffine = runSMTWith z3Timeout5 $ do
  (a,b) <- affineParameters
  constrain $ affineAt a b 0 .== targetAt 0
  query $ let loop = do
                result <- checkSat
                case result of
                  Sat -> do
                    candidate <- (,) <$> getValue a <*> getValue b
                    io $ putStrLn $ "Candidate: " ++ show candidate
                    witness <- io $ candidateCounterexample candidate
                    case witness of
                      Nothing -> pure candidate
                      Just x  -> do
                        io $ putStrLn $ "Counterexample: " ++ show x
                        constrain $ affineAt a b (literal x) .== targetAt (
    literal x)
                        loop
                Unsat -> io $ fail "no program in this template matches
    the samples"
                _     -> io $ fail "candidate search was inconclusive"
          in loop
```

`io` runs the verification helper in a separate solver session and returns its ordinary Haskell result to the candidate query. A refuting input is lifted back with `literal` and added as a permanent constraint. In the checked run, the first candidate is $(-3, 1)$, the verifier returns -1, and the next candidate is $(3, 1)$; the final check is unsatisfiable. Candidate and counterexample order are solver choices, not part of the algorithm's contract.

Each counterexample excludes the current candidate and never excludes a correct one. With 49 candidates, the loop has finitely many refinements provided every solver call returns a conclusive answer. An unbounded grammar would lose this termination argument. Here candidate constraints use concrete sample inputs, and verification uses concrete coefficients,

so each call avoids multiplying two unknown integer quantities even though the combined synthesis problem mentions ax.

The companion checks the solution against an enumeration of the entire template space and re-evaluates concrete expressions independently. The universal verification call remains essential: agreement on tested inputs alone is not the acceptance condition. This is counterexample-guided inductive synthesis (Solar-Lezama et al. 2006); the "inductive" step here learns from examples and is distinct from the structural induction proofs introduced next.

15 Proofs by induction

SMT solvers decide many theories well, but an induction argument may be needed to connect recursive definitions to a theorem. SBV's TP layer (in `Data.SBV.TP`) lets the programmer write the argument in Haskell and has the solver check each step. The programmer decides *what* to prove and *how* to break it into steps; the solver decides whether each step is valid.

This division connects symbolic execution to calculational reasoning. We choose intermediate expressions and useful hypotheses; the solver checks the logical consequences of each step. The notation below makes the proof obligations explicit. Section 25.2 relates this style to equational reasoning in proof assistants and refinement-based proofs.

15.1 A first inductive proof

We return to the property that eluded `prove` in Section 8:

```
sumNonNeg :: TP (Proof (Forall "xs" [Integer] -> SBool))
sumNonNeg = induct "sumNonNeg"
  (\(Forall xs) -> SL.all (.>= 0) xs .=> sumL xs .>= 0) $
  \ih (x, xs) -> [SL.all (.>= 0) (x .: xs)]
              |- sumL (x .: xs) .>= 0
              =: x + sumL xs .>= 0
              ?? ih
              =: sTrue
              =: qed
```

The type `Proof (Forall "xs" [Integer] -> SBool)` records quantification over integer lists; the predicate is supplied to `induct`. The name `xs` identifies the variable for output and instantiation. Scripts live in the `TP` monad: `runTP` checks their obligations and returns a proof object; Haskell typechecking alone does not establish the theorem.

The function `induct` applies list induction: SBV checks the base case automatically, then supplies the hypothesis `ih` for `xs` to prove the property for `x .: xs`. This covers finite lists; the scope for lists and recursive datatypes is discussed in Section 23.2.

The step is a *calculation*. Its first line lists the assumptions, here the precondition for the longer list, and `|-` starts a chain of expressions connected by `=:`. Each link must be justified by the definitions, the assumptions, and any hints supplied with `??`. The chain ends with `qed`. A hint written between two expressions justifies the transition *to the following expression*; `?? ih` is not a new assumption about the entire theorem. In this example the expressions being equated are Boolean formulas, and the calculation reduces the target formula to `sTrue`. Running the proof shows each checked step (the final proof-object display is omitted here):

```
ghci> runTP sumNonNeg
Inductive lemma: sumNonNeg
  Step: Base                 Q.E.D.
  Step: 1                    Q.E.D.
  Step: 2                    Q.E.D.
  Result:                    Q.E.D.
Functions proven terminating: sbv.foldr, sumL
```

Step 1 unfolds `sumL` once. Step 2 uses the hypothesis: `x` is nonnegative by assumption and so is `sumL xs`, so the sum is too. The `Result` line checks that the chain, together with the base case, establishes the theorem as stated. The last line reports that the recursive functions used, including the library's `foldr` that underlies `SL.all`, were checked for termination. Section 23.1 explains how TP enforces this requirement.

15.2 Associativity of append

Before introducing accumulators, consider a structural property of lists:

$$(xs \mathbin{+\!\!+} ys) \mathbin{+\!\!+} zs = xs \mathbin{+\!\!+} (ys \mathbin{+\!\!+} zs).$$

Appending recurses on its first argument, so we induct on `xs`, keeping `ys` and `zs` universally quantified. Only the grouping changes; the order of the elements stays the same.

The base case is immediate: replacing `xs` by `[]` reduces both sides to `ys SL.++ zs`. In the inductive case, write the first list as `x .: xs`. Unfolding append twice exposes an equality about the tail, where the induction hypothesis applies:

```
appendAssoc :: TP (Proof (Forall "xs" [Integer] -> Forall "ys" [Integer]
                       -> Forall "zs" [Integer] -> SBool))
appendAssoc = induct "appendAssoc"
  (\(Forall xs) (Forall ys) (Forall zs) ->
      (xs SL.++ ys) SL.++ zs .== xs SL.++ (ys SL.++ zs)) $
  \ih (x, xs) ys zs ->
    [] |- ((x .: xs) SL.++ ys) SL.++ zs
       =: (x .: (xs SL.++ ys)) SL.++ zs
       =: x .: ((xs SL.++ ys) SL.++ zs)
       ?? ih `at` (Inst @"ys" ys, Inst @"zs" zs)
       =: x .: (xs SL.++ (ys SL.++ zs))
       =: (x .: xs) SL.++ (ys SL.++ zs)
       =: qed
```

The first two links use `(x .: xs) SL.++ ys = x .: (xs SL.++ ys)`. The third uses the hypothesis `(xs SL.++ ys) SL.++ zs .== xs SL.++ (ys SL.++ zs)` beneath the outer cons. The expression `ih `at` (Inst @"ys" ys, Inst @"zs" zs)` specializes its two quantified parameters to the current lists; the tail `xs` is already fixed by induction. The last link folds the definition of append back into the required right-hand side.

Running `runTP appendAssoc` checks the base case, these four links, and the final result. The proof needs no preconditions, hence the empty list before `|-`. Its hypothesis has exactly the shape exposed by unfolding the definition. Accumulator programs will require us to work harder to obtain that match.

15.3 Solver obligations in a calculation

Figure 1 shows the queries under assumptions A, with facts H_i supplied by hints h_i. Definitions are available in every query. SBV also checks that the chain establishes the stated theorem under its precondition. Boolean hints must themselves follow from the local assumptions; Section 19.3 shows what happens when one is wrong. For failed steps, see Section 24.1.

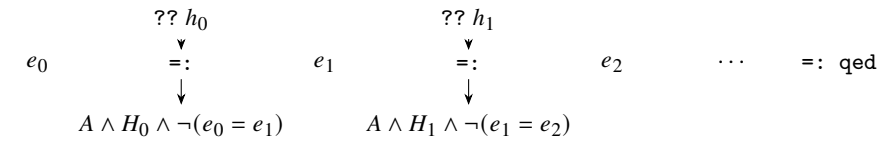


Fig. 1. Each hint supplies a fact for the next equality; its query must be unsatisfiable.

15.4 Generalization over accumulators

Reversing a list with an accumulator is a standard optimization:

```
revAcc :: SList Integer -> SList Integer -> SList Integer
revAcc = smtFunction "revAcc" $ \acc xs -> [sCase| xs of
            []     -> acc
            x : xt -> revAcc (x .: acc) xt
         |]
```

We expect `revAcc [] xs` to equal `SL.reverse xs`. The direct attempt is instructive:

```
revWeak :: TP (Proof (Forall "xs" [Integer] -> SBool))
revWeak = induct "revWeak"
  (\(Forall xs) -> revAcc [] xs .== SL.reverse xs) $
  \ih (x, xs) -> [] |- revAcc [] (x .: xs)
                    =: revAcc [x] xs
                    ?? ih
                    =: SL.reverse xs SL.++ [x]
                    =: SL.reverse (x .: xs)
                    =: qed
```

Run it with a five-second per-call timeout (intermediate output omitted):

```
ghci> runTPWith z3Timeout5 revWeak
...
*** Failed to prove revWeak.2.
```

Step 2 tries to use the induction hypothesis, but the hypothesis is about `revAcc [] xs`, while the expression contains `revAcc [x] xs`. After one step of recursion, the accumulator is no longer empty, and the hypothesis says nothing about it. The timeout leaves the statement unresolved; it does not refute reversal correctness.

The fix is the oldest trick in inductive proofs: prove a more general statement. The theorem must describe `revAcc` for *every* accumulator, so that the hypothesis can be used at whatever accumulator the recursion produces:

```
revAccCorrect :: TP (Proof (Forall "xs" [Integer] -> Forall "acc" [Integer]
     -> SBool))
```

```
revAccCorrect = induct "revAccCorrect"
  (\(Forall xs) (Forall acc) -> revAcc acc xs .== SL.reverse xs SL.++ acc)
    $
  \ih (x, xs) acc -> [] |- revAcc acc (x .: xs)
                        =: revAcc (x .: acc) xs
                        ?? ih
                        =: SL.reverse xs SL.++ (x .: acc)
                        =: (SL.reverse xs SL.++ [x]) SL.++ acc
                        =: SL.reverse (x .: xs) SL.++ acc
                        =: qed
```

Here `acc` is universally quantified, and the induction hypothesis `ih` holds for all accumulators. When it is supplied as a hint, the solver instantiates it with `x .: acc`. The remaining steps use ordinary laws of `++` and `reverse`, using their sequence operations and definitions. The original statement is the special case `acc = []`:

```
reverseCorrect :: TP (Proof (Forall "xs" [Integer] -> SBool))
reverseCorrect = do
  generalized <- recall revAccCorrect
  lemma "reverseCorrect"
    (\(Forall xs) -> revAcc [] xs .== SL.reverse xs)
    [proofOf generalized]
```

`recall` obtains the generalized proof, reusing it if it is already cached in the current TP run. `proofOf` packages that proof as a dependency for `lemma`, which asks the solver to establish the specialization. This separates the reusable inductive result from the interface property callers need.

Exercise 5. Prove that the length of `revAcc acc xs` equals `SL.length xs + SL.length acc`. Why must the accumulator remain universally quantified in the induction hypothesis, even when the intended caller starts with `[]`? Solution: `revAccLength`.

16 Kadane's algorithm and invariant discovery

The maximum segment sum problem asks for the largest sum of a contiguous segment of a list, where the empty segment has sum 0. For `[1, -2, 3, 4, -1, 2]`, the answer is 8, from `[3, 4, -1, 2]`. Kadane's algorithm (Bentley 1984, Algorithm 4) solves it in a single left-to-right pass. Its correctness is not obvious, which makes it a good example of finding and proving an invariant.

16.1 Specification

We specify the problem by its most direct recursive reading:

```
mss :: SList Integer -> SInteger
mss = smtFunction "mss" $ \xs -> [sCase| xs of
         []    -> 0
         _ : t -> mssBegin xs `smax` mss t
      |]

mssBegin :: SList Integer -> SInteger
```

```
mssBegin = smtFunction "mssBegin" $ \xs -> [sCase| xs of
              []    -> 0
              h : t -> 0 `smax` (h `smax` (h + mssBegin t))
           |]
```

`mssBegin xs` is the best sum of a segment that starts at the beginning of `xs`: either the empty segment (0), just the head, or the head followed by the best segment starting at the tail. `mss xs` is the best of `mssBegin xs` and the best segment of the tail: a segment either starts at the head or lies entirely in the tail. With unit-cost arithmetic and without memoizing calls across suffixes, these recurrences take quadratic time. Since they are symbolic functions, they also run on literal lists, which is a convenient way to test a specification before reasoning about it: `mss [1, -2, 3, 4, -1, 2]` evaluates to 8.

To check that the recurrence matches the intended problem, we can independently enumerate segments with ordinary Haskell's `Data.List.inits` and `Data.List.tails`:

```
segmentOracle :: [Integer] -> Integer
segmentOracle xs = maximum [sum segment | suffix <- tails xs, segment <-
    inits suffix]
```

Every segment is a prefix of a suffix, and the empty segment ensures the maximum is defined even for `[]`. The companion compares both `mss` and `kadane` against this oracle on all 3,906 lists of length at most five over $\{-2, -1, 0, 1, 2\}$, plus the examples above. These tests check the connection to the intended specification; the theorem below establishes equivalence for every finite integer list.

Bird (1989) derives the linear algorithm from a segment-enumerating specification using algebraic laws for higher-order functions, including Horner's rule, the scan lemma, and fold–scan fusion. The resulting paired fold maintains the same two quantities as our implementation. We expect a faithful TP rendition of that compact derivation to require considerably more proof infrastructure than the invariant proof below: laws must be instantiated, their side conditions discharged, and any missing supporting lemmas proved. TP can express such arguments—its list examples already include fusion lemmas, and Section 20 develops a related higher-order law—but the solver does not automatically supply this algebra of program transformations. We have not mechanized Bird's derivation here. Our proof establishes the accumulator implementation's equivalence to a recursive specification; it does not derive the implementation from the segment enumeration. Section 26.2 proposes that derivation as a further project.

16.2 Implementation

Kadane's algorithm maintains two accumulators while it scans the list: `meh`, the maximum sum of a segment ending at the current position, and `msf`, the maximum found so far:

```
kadane :: SList Integer -> SInteger
kadane xs = kadaneHelper xs 0 0

kadaneHelper :: SList Integer -> SInteger -> SInteger -> SInteger
kadaneHelper = smtFunction "kadaneHelper" $ \xs meh msf -> [sCase| xs of
                 []    -> msf
                 h : t -> let meh' = 0 `smax` (h + meh)
```

```
                                  msf' = msf `smax` meh'
                                in kadaneHelper t meh' msf'
            |]
```

At each element `h`, the best segment ending here either extends the best segment ending at the previous position or is empty, so the new `meh` is `0 `smax` (h + meh)`. The best so far is updated if that value is larger. The allowance of an empty segment matters: an all-negative input returns 0. An API that requires a nonempty segment would need a different specification and initialization.

16.3 Invariant discovery

As with reversal, the theorem we want, `kadane xs .== mss xs`, concerns the initial accumulators `0` and `0`. An inductive proof will reach `kadaneHelper` with other accumulator values, so we must describe what `kadaneHelper xs meh msf` computes for *any* sensible `meh` and `msf`.



Fig. 2. Three candidates for the maximum segment sum: within the processed prefix, within the remaining suffix, or across the boundary. Segments may be empty; each label gives the best sum for that case.

Split the list into a processed prefix and the remaining suffix `xs`. Figure 2 shows the three possible locations of the best segment. The crossing candidate combines the best suffix of the processed prefix (`meh`) with the best prefix of `xs` (`mssBegin xs`).

For example, split `[1,-2,3,4,-1,2]` after the 4. The processed prefix has $meh = 7$ and $msf = 7$, while the suffix has $mss = 2$ and $mssBegin = 1$. The three candidates are 7, 2, and $7 + 1 = 8$; the best segment crosses the boundary. So the invariant is

$$kadaneHelper(xs, meh, msf) = \max\big(msf,\ mss(xs),\ meh + mssBegin(xs)\big),$$

under the preconditions that make the accumulators meaningful: $meh \geq 0$ (the empty segment is always available) and $msf \geq meh$ (the best segment so far is at least as good as the best one ending here). Without the preconditions the statement is false: for an empty `xs`, the right side would include meh even when it exceeds msf.

16.4 The proof

Once the invariant is written down, the proof is short:

```
kadaneCorrect :: TP (Proof (Forall "xs" [Integer] -> SBool))
kadaneCorrect = do
  inv <- induct "kadaneInvariant"
           (\(Forall xs) (Forall meh) (Forall msf) ->
```

```
                    (meh .>= 0 .&& msf .>= meh)
                .=> kadaneHelper xs meh msf
                    .== msf `smax` mss xs `smax` (meh + mssBegin xs)) $
          \ih (a, as) meh msf ->
             [meh .>= 0, msf .>= meh]
          |- let meh' = 0 `smax` (a + meh)
                 msf' = msf `smax` meh'
             in kadaneHelper (a .: as) meh msf
             =: kadaneHelper as meh' msf'
             ?? ih `at` (Inst @"meh" meh', Inst @"msf" msf')
             =: msf' `smax` mss as `smax` (meh' + mssBegin as)
             =: qed

  calc "kadaneCorrect" (\(Forall xs) -> kadane xs .== mss xs) $
       \xs -> let z = 0 :: SInteger
              in [] |- kadane xs
                    =: kadaneHelper xs z z
                    ?? inv `at` (Inst @"xs" xs, Inst @"meh" z, Inst @"msf" z
    )
                    =: z `smax` mss xs `smax` (z + mssBegin xs)
                    =: mss xs `smax` mssBegin xs
                    =: mss xs
                    =: qed
```

The inductive step unfolds `kadaneHelper` once and applies the hypothesis at the new accumulators. Here the hint is more specific than in the earlier proofs: `ih `at` (Inst @"meh" meh', Inst @"msf" msf')` instantiates the quantified accumulators explicitly. The solver could search for the instantiation, but stating it makes the step easier and documents the argument. The remaining equality between the instantiated hypothesis and the invariant for `a .: as` is arithmetic over `smax`, which the solver checks without further help, using the definitions of `mss` and `mssBegin`. The preconditions are needed to instantiate the hypothesis: the new accumulators satisfy them because `meh'` is a maximum with 0 and `msf'` is a maximum with `meh'`.

The main theorem then follows by a calculation without induction. With both accumulators 0, the invariant gives $\max(0, mss(xs), mssBegin(xs))$, and the solver discharges the last two steps: `mss` is nonnegative and at least `mssBegin` by its definition. Abbreviated output shows the two results:

```
ghci> runTP kadaneCorrect
Inductive lemma: kadaneInvariant
  ...
  Result:                             Q.E.D.
Lemma: kadaneCorrect
  ...
  Result:                             Q.E.D.
```

16.5 Lessons

The effort in this proof went into one place: the invariant. Once we had explained *why* the algorithm works, as the three-way case analysis on where the best segment lies, the mechanical checking was immediate. This division of labor is typical of TP. The solver handles unfolding definitions, arithmetic, and case analysis on `smax`; the human supplies generalizations and intermediate statements. When a proof gets stuck, the useful question is rarely "which tactic is missing?" and usually "what is the statement that explains this step?"

Exercise 6. *Challenge.* Require the chosen segment to be nonempty. Write a specification and a linear-time algorithm, and prove their equivalence under an explicit nonempty-input precondition. Test all-negative lists. Solutions: `mssNonEmpty`, `kadaneNonEmpty`, and `kadaneNonEmptyCorrect`. The independent segment enumerator is `segmentNonEmpty Oracle`. The solutions return 0 on `[]`, outside the theorem's contract.

17 Induction on decreasing measures

The next two examples use `sInduct` to choose a measure suited to the computation: the size of an expression tree, then the length of a search interval.

17.1 Expression trees

Return to `Expr`: mirroring swaps the children at every addition and multiplication node, preserving evaluation because both operations commute.

```
mirror :: SExpr -> SExpr
mirror = smtFunction "mirror" $ \e -> [sCase| e of
  Num n   -> sNum n
  Add a b -> sAdd (mirror b) (mirror a)
  Mul a b -> sMul (mirror b) (mirror a)
  |]
```

Each binary case needs the hypothesis for both children before commutativity applies. The calculational `induct` interface supports integers and lists, so for this datatype we use `sInduct` with a size measure. Reuse `exprSize` and its nonnegativity theorem `exprSizePos` from Section 10.1. That theorem uses `inductiveLemma` to apply the generated datatype induction principle without further guidance. Here `exprSizeSmaller` uses this fact to show that each child is smaller than its parent. These facts justify applying the hypothesis to each child. The listing abbreviates the identical multiplication case; the companion checks all three cases in full:

```
mirrorCorrect :: TP (Proof (Forall "e" Expr -> SBool))
mirrorCorrect = do
  pos <- recall exprSizePos
  smaller <- recall exprSizeSmaller
  sInduct "mirrorCorrect" (\(Forall e) -> eval (mirror e) .== eval e)
    (exprSize, [proofOf pos]) $ \ih e -> [] |- [pCase| e of
      Num n -> eval (mirror (sNum n)) =: eval (sNum n) =: qed
      Add a b -> eval (mirror (sAdd a b))
              =: eval (mirror b) + eval (mirror a)
              ?? smaller `at` (Inst @"a" a, Inst @"b" b)
```

```
            ?? ih `at` Inst @"e" a
            ?? ih `at` Inst @"e" b
            =: eval b + eval a
            =: eval a + eval b
            =: eval (sAdd a b)
            =: eval e
            =: qed
    Mul a b -> -- as for Add, with (*) in place of (+)
    |]
```

The hypothesis supplied by `sInduct` is conditional: it proves the result for an expression only if its measure is smaller than that of the current `e`. Each `smaller` hint discharges these conditions; the two `ih` hints instantiate the hypothesis separately at `a` and `b`. After substituting their evaluations, commutativity finishes the case. The final `eval e` reconnects each binary case to the theorem's goal. The `pCase` quasi-quoter splits the *proof* by constructor and checks that the cases cover the datatype; `sCase` defines symbolic functions. The `Num` case has no recursive children and needs no hypothesis. Running `runTP mirrorCorrect` checks all three cases and returns `Proven`. The size argument is structural induction expressed through `sInduct`'s more general principle of induction on a decreasing measure.

17.2 Binary search with a decreasing interval

Section 3.2 checked midpoint arithmetic. We now ask whether the search itself finds a target in a sorted interval. SBV's `TP.BinarySearch` example uses `Arr = SArray Integer Integer` and `Idx = (SInteger, SInteger)`, with both bounds inclusive:

```
bsearch :: Arr -> Idx -> SInteger -> SMaybe Integer
bsearch array (low, high) = f array low high
  where f = smtFunctionWithMeasure "bsearch" (\_arr lo hi _x -> (hi - lo + 1)
     `smax` 0, [])
          $ \arr lo hi x ->
               let mid  = (lo + hi) `sEDiv` 2
                   xmid = arr `readArray` mid
               in [sCase| lo of
                     _ | lo .> hi   -> sNothing
                     _ | xmid .== x -> sJust mid
                     _ | xmid .< x  -> bsearch arr (mid+1, hi)    x
                     _              -> bsearch arr (lo,    mid-1) x
                  |]
```

Here `sEDiv` by 2 computes the floor of the integer average, including for negative bounds. The measure $\max(hi - lo + 1, 0)$ is the number of indices remaining. On a nonempty interval, the midpoint lies within its bounds, so either recursive call strictly decreases this measure. The empty interval returns `sNothing`; a match returns `sJust mid`.

The companion imports this module as `Search` and `Data.SBV.Maybe` as `SM`. We state the result contract explicitly:

```
searchFound :: Search.Arr -> Search.Idx -> SInteger
            -> SMaybe Integer -> SBool
searchFound arr (lo, hi) x result =
  SM.isJust result .&& lo .<= i .&& i .<= hi .&& readArray arr i .== x
```

```
  where i = SM.fromJust result

searchContract :: Search.Arr -> Search.Idx -> SInteger
               -> SMaybe Integer -> SBool
searchContract arr bounds x result =
  ite (Search.inArray arr bounds x)
      (searchFound arr bounds x result)
      (SM.isNothing result)
```

The predicate `Search.inArray` means that the target occurs at some index between the bounds. If it occurs, `searchFound` requires a `Just` result, an index within the interval, and the correct value at that index. Otherwise the result must be `Nothing`. The theorem assumes that the array is nondecreasing throughout the interval; duplicates are allowed, and no particular matching index is prescribed.

The proof `binarySearchCorrect` uses `sInduct` with the same interval measure. Its two helper lemmas show that restricting the search to the selected half preserves sortedness and whether the target occurs. In the upper-half case, for example, all values at or below the midpoint are too small. The calculation supplies that range lemma and instantiates the induction hypothesis at $[mid + 1, hi]$; the lower-half case is symmetric. Empty intervals and immediate matches close directly. Running `binarySearchProof` checks this argument with CVC5, including termination of `bsearch`; the complete calculation is in the companion.

This proof uses unbounded integer indices, so `lo + hi` cannot overflow. It complements the earlier arithmetic proof, but does not verify a fixed-width implementation of the whole search: that further step must also justify the index conversions and updates `mid+1` and `mid-1`.

18 McCarthy 91 and termination contracts

Nested recursion can require knowing what an inner call returns before showing that the outer call decreases the measure. McCarthy's 91 function is a classic example. Here is its SBV definition, extracted from `Documentation.SBV.Examples.TP.McCarthy91`:

```
mcCarthy91 :: SInteger -> SInteger
mcCarthy91 = smtFunctionWithContract "mcCarthy91"
               ( \n -> 0 `smax` (101 - n)
               , \n r -> n .<= 100 .=> r .== 91
               , []
               )
           $ \n -> [sCase| n of
                       _ | n .> 100 -> n - 10
                       _            -> mcCarthy91 (mcCarthy91 (n + 11))
                   |]
```

For inputs above 100 it returns immediately; otherwise, the result of one recursive call becomes the input to another. The triple passed to `smtFunctionWithContract` contains a measure, a postcondition on the input `n` and result `r`, and a list of helper lemmas (empty here). The measure is $\max(0, 101 - n)$, and the postcondition says that inputs at most 100 return 91. SBV checks the postcondition together with the measure decrease by well-founded induction; the contract is not an additional axiom. Its availability at smaller measures justifies the nested call.

To see why the contract matters, consider $n \leq 100$. The inner call at $n + 11$ has a strictly smaller measure. If $n \leq 89$, its input is at most 100, so its checked postcondition gives the result 91. The outer call then has measure 10, strictly below $101 - n$. If $90 \leq n \leq 100$, the inner call takes the nonrecursive branch and returns $n + 1$, so the outer call's measure decreases by one. The numerical argument grows in both cases; the measure records progress toward the nonrecursive branch instead of requiring arguments themselves to shrink.

The complete specification also covers inputs above 100:

```
spec91 :: SInteger -> SInteger
spec91 n = ite (n .> 100) (n - 10) 91
```

The library's `correctness` proof establishes `mcCarthy91 n .== spec91 n` for every integer. Two helper lemmas, `case1` and `case2`, cover $n \geq 100$ and $90 \leq n \leq 100$. For $n < 90$, the guided strong-induction step is:

```
  case3 <- sInduct "case3"
                   (\(Forall n) -> n .< 90 .=> mcCarthy91 n .== spec91 n)
                   (\n -> abs (101 - n), []) $
                   \ih n -> [n .< 90] |- mcCarthy91 n
                                      ?? "unfold"
                                      =: mcCarthy91 (mcCarthy91 (n + 11))
                                      ?? ih    `at` Inst @"n" (n + 11)
                                      ?? case2 `at` Inst @"n" (n + 11)
                                      =: mcCarthy91 91
                                      =: qed
```

Here `sInduct` uses $|101 - n|$ as the proof measure. The recursive function's admissibility check uses the clipped measure in its definition; these measures agree on the inputs below 90 considered in this case. After unfolding, the proof instantiates the hypothesis at $n+11$. When that input is still below 90, the hypothesis gives the inner result 91; otherwise `case2` supplies it. The remaining expression is `mcCarthy91 91`, which unfolds to 91, as does `spec91 n` under $n < 90$. A final `cases` calculation combines the three ranges and checks that they cover every input.

The companion imports the module as `McCarthy`. Running `McCarthy.correctness` checks the proof and reports `Functions proven terminating: mcCarthy91`, followed by the `Proven` result. Run it separately with `cabal run tutorial-examples -- --mccarthy91`. Validation also checks that the returned proof has no unchecked dependencies. The neighboring `Ackermann` module gives further examples of nested recursion and lexicographic decrease.

19 Failed TP proofs and counterexamples

A failed TP run is a diagnostic about an obligation, not a verdict on every statement in the script. A solver can refute the stated theorem, refute an intermediate equality, or leave an obligation unresolved. The examples below make these outcomes concrete; the companion runs the expected failures and checks their diagnostics. The first example adapts `Documentation.SBV.Examples.TP.Basics`.

19.1 Concrete inputs and observed values

Suppose a length function returns 42 whenever its input has more than five elements and contains 42. This intentionally incorrect definition is compared with the library length function:

```
badLength :: SList Integer -> SInteger
badLength xs = ite (SL.length xs .> 5 .&& 42 `SL.elem` xs)
                   42 (SL.length xs)

badLengthProof :: TP (Proof (Forall "xs" [Integer] -> SBool))
badLengthProof = lemma "badLengthProof"
  (\(Forall xs) -> observe "imp"  (badLength xs)
              .== observe "spec" (SL.length xs)) []
```

`observe` names expressions to include in the model; it does not add a premise to the theorem. Running `runTPWith z3Timeout5 badLengthProof` produces a diagnostic followed by a `Failed` exception. Here is the validated diagnostic with its spelling updated to `Counterexample`:

```
Lemma: badLengthProof
*** Failed to prove badLengthProof.
Falsifiable. Counterexample:
  xs   = [11,13,16,18,21,42] :: [Integer]
  imp  =                 42 :: Integer
  spec =                  6 :: Integer
```

The universal input `xs` has become a concrete finite list. The observations explain the disagreement: the implementation returns 42, while the specification returns 6. The elements other than 42 have no particular significance; another solver run may choose different ones. Evaluate both sides on the reported input before deciding whether the implementation or the specification needs repair. Validation checks the displayed model with ordinary list length and membership. TP does not return a `Proof` for a refuted obligation. Its validation driver catches the exception because failure is expected in this example.

19.2 A failed step in a true theorem

A calculation can fail even when its advertised theorem is true:

```
badCalculation :: TP (Proof (Forall "n" Integer -> SBool))
badCalculation = calc "badCalculation"
  (\(Forall n) -> n + n .== 2 * n) $ \n ->
    [] |- n + n
       =: 2 * n + 1
       =: 2 * n
       =: qed
```

The theorem is the identity $n + n = 2n$, but the first step incorrectly inserts 1. For `runTPWith z3Timeout5 badCalculation`, the relevant output is:

```
*** Failed to prove badCalculation.1.
Falsifiable. Counterexample:
  n = 0 :: Integer
```

This refutes the first equality, $n + n = 2n + 1$: its two sides are 0 and 1 at the reported input. It does not refute $n + n = 2n$, which the companion proves separately. The suffix `.1` identifies the failing obligation. Remove the incorrect step and the calculation can establish its intended theorem.

More generally, the model satisfies the assumptions and hints available to that step and falsifies its goal. A missing premise, a missing lemma, or an induction hypothesis instantiated at the wrong arguments can leave an obligation too weak. A model of such an obligation need not describe a reachable program execution. Inspect the local goal and its premises before treating it as a bug in the original program. Conversely, a timeout or `unknown` supplies no refuting model: the `revWeak` attempt in Section 15 is unresolved, not disproved. Termination-check failures concern admissibility of a definition and are a further, separate diagnostic (Section 23.1).

19.3 An incorrect justification

A Boolean expression supplied with `??` is itself a proof obligation. SBV first checks that it follows from the local assumptions, before using it to justify the equality. For example, the following calculation inserts the unjustified claim that its arbitrary integer input is nonnegative:

```
badJustification :: TP (Proof (Forall "n" Integer -> SBool))
badJustification = calc "badJustification"
  (\(Forall n) -> n + n .== 2 * n) $ \n ->
    [] |- n + n
       ?? (n .>= 0)
       =: 2 * n
       =: qed
```

Both the stated theorem and the equality are valid, but the hint is not. To see the separate check of Boolean hints, enable `printAsms`:

```
showHintFailure :: IO ()
showHintFailure = void $ runTPWith
  z3Timeout5{tpOptions = (tpOptions z3Timeout5){printAsms = True}}
  badJustification
```

Running `showHintFailure` produces:

```
Lemma: badJustification
  Asms: 1
*** Failed to prove assumptions for badJustification.1.
Falsifiable. Counterexample:
  n = -1 :: Integer
```

At this input the proposed justification is false, although the equality still holds: both sides are -2. The words *assumptions for* distinguish this failure from a refuted calculation step. Here the repair is simply to remove the hint. If a proof genuinely needs a nonnegative input,

that restriction must be justified by the surrounding context or made a precondition of the theorem; inserting it with `??` does not establish it. Unlike a previously proved lemma supplied as a hint, a fresh Boolean hint must be discharged at its point of use. In the pinned SBV version these checks are normally quiet, including their failure diagnostics; `printAsms = True` exposes the counterexample. Validation checks that the reported integer refutes the hint while satisfying the equality.

19.4 Function interpretations as counterexamples

A model can also describe an uninterpreted operation. The library example `forall DisjunctionNot` attempts the invalid equivalence

$$((\forall x.\ p(x)) \vee (\forall x.\ q(x))) \quad \Longleftrightarrow \quad \forall x.\ (p(x) \vee q(x)).$$

With `p` and `q` uninterpreted predicates on integers, TP reports:

```
p :: Integer -> Bool
p 3 = True
p _ = False

q :: Integer -> Bool
q 3 = False
q _ = True
```

These are finite descriptions of total functions: the underscore gives the value at every unlisted input. At each integer at least one predicate is true, but neither predicate is true at every integer. Thus the right side holds and the left side fails. The exceptional integer need not be 3 in another run. The companion invokes the library example and checks this interpretation of its model. Models of functions, arrays, and datatypes should be read according to their symbolic domains, just as scalar counterexamples are.

20 Polymorphic and higher-order proofs

Proof scripts can be reused across types and parameterized by operations. The next examples show what each form of reuse establishes.

20.1 Polymorphic scripts, monomorphic proofs

Reversal preserves length regardless of the element type. We can express the proof once, using Haskell polymorphism:

```
reverseLength :: forall a. SymVal a => TP (Proof (Forall "xs" [a] -> SBool)
    )
reverseLength = induct "reverseLength"
  (\(Forall xs) -> SL.length (SL.reverse xs) .== SL.length xs) $
  \ih (x, xs) -> [] |- SL.length (SL.reverse (x .: xs))
                    =: SL.length (SL.reverse xs SL.++ [x])
                    =: SL.length (SL.reverse xs) + 1
                    ?? ih
                    =: SL.length xs + 1
                    =: SL.length (x .: xs)
                    =: qed
```

The Haskell `forall a` makes the *script* polymorphic; `SymVal a` provides the symbolic representation of its elements. Before running it, we choose a monotype. For example, in GHCi:

```
runTP (reverseLength @Integer)
runTP (reverseLength @Word8)
```

Each invocation checks a separate theorem over that type's solver sort. The companion runs both.

This differs from `Forall "xs" [a]`, which quantifies over *values* inside the theorem after `a` has been chosen. A successful integer instance does not by itself certify every type instance of the script: SBV is not quantifying over Haskell types. Polymorphism lets us reuse the argument and its implementation, while each proof run establishes a monomorphic result. This distinction matters especially for arithmetic laws, whose truth may change between `Integer` and `Word8`.

20.2 The bookkeeping law

Many useful list laws concern an operation supplied by the caller. The *bookkeeping law*, adapted here from `Documentation.SBV.Examples.TP.Lists`, says that reducing a concatenation equals reducing each chunk and then reducing those results:

$$reduce(concat\ xss) = reduce(map\ reduce\ xss), \qquad reduce = foldr\ f\ e.$$

Associativity of f and a two-sided identity e suffice. For integer addition with identity 0, this is "sum the elements" versus "sum the chunk sums." Commutativity is not assumed: the order of elements is preserved. Both the outer list `xss` and every chunk are finite; the law makes no claim about consuming an infinite stream of chunks.

We reuse a library lemma with the following import:

```
import qualified Documentation.SBV.Examples.TP.Lists as ListProofs
```

The proof has two dependencies: `foldrOverAppend`, and a helper `foldBase` proved in the companion. Their conclusions are, respectively,

$$foldr\ f\ e\ (xs\ ++\ ys) = foldr\ f\ (foldr\ f\ e\ ys)\ xs,$$
$$foldr\ f\ y\ xs = f\ (foldr\ f\ e\ xs)\ y.$$

The first needs no assumptions on f. The second requires associativity and the left-identity law. Its empty-list case uses $f\ e\ y = y$; its inductive step uses associativity to move y past the head. These lemmas expose exactly the two transformations the main induction needs:

```
bookKeeping :: forall a. SymVal a
            => SBV a -> (SBV a -> SBV a -> SBV a)
            -> TP (Proof (Forall "xss" [[a]] -> SBool))
bookKeeping e f = do
  let assoc = quantifiedBool $ \(Forall x) (Forall y) (Forall z) ->
                f x (f y z) .== f (f x y) z
      rUnit = quantifiedBool $ \(Forall x) -> f x e .== x
      lUnit = quantifiedBool $ \(Forall x) -> f e x .== x
      reduce = SL.foldr f e
  base <- foldBase e f
  over <- ListProofs.foldrOverAppend e f
```

```
induct "bookKeeping"
  (\(Forall xss) -> assoc .&& rUnit .&& lUnit .=>
     reduce (SL.concat xss) .== reduce (SL.map reduce xss)) $
  \ih (xs, xss) -> [assoc, rUnit, lUnit] |- reduce (SL.concat (xs .: xss)
  )
    =: reduce (xs SL.++ SL.concat xss)
    ?? over
    =: SL.foldr f (reduce (SL.concat xss)) xs
    ?? ih
    =: SL.foldr f (reduce (SL.map reduce xss)) xs
    ?? base `at` (Inst @"xs" xs, Inst @"y" (reduce (SL.map reduce xss)))
    =: f (reduce xs) (reduce (SL.map reduce xss))
    =: reduce (reduce xs .: SL.map reduce xss)
    =: reduce (SL.map reduce (xs .: xss))
    =: qed
```

`quantifiedBool` packages a quantified property as an `SBool`, so the algebraic laws can appear as assumptions in the calculation and as the premise of the theorem. They are not unconditionally added as user axioms. Induction is over the outer list of chunks: unfold `concat`, move the fold across append, use the induction hypothesis on the remaining chunks, then use `foldBase` to combine their result with that of the first chunk.

The left-identity assumption has a concrete purpose. Take $f\ x\ y = x$ and $e = 0$. This operation is associative and has right identity 0, but for `xss = [[],[1]]`, the two sides of the bookkeeping law are 1 and 0. The empty first chunk contributes an identity that must work on the left. The companion checks this counterexample as well as the proof.

20.3 An arbitrary operation at a fixed type

The parameters `e` and `f` are Haskell arguments to the proof script. To prove the law without choosing an implementation of the operation, give them fresh uninterpreted names:

```
bookKeepingAbstract :: IO (Proof (Forall "xss" [[Integer]] -> SBool))
bookKeepingAbstract = runTP $
  bookKeeping @Integer (uninterpret "unit") (uninterpret "combine")
```

An `uninterpret` declaration supplies a logical name without defining equations. Here `combine` is a total function on integers; the proof must work for every interpretation satisfying the stated algebraic laws. The solver does not enumerate implementations of `combine`. SBV specializes its list operations to this logical operation and reasons about the resulting definitions. The higher-order Haskell interface thus supports a proof about an arbitrary operation at a fixed type. We do not pass a Haskell function as an ordinary `Forall` argument of the theorem.

Passing `0` and `(+)` instead gives a concrete instance. The companion also checks that instance and derives the unconditional sum-of-chunk-sums law. An uninterpreted operation represents a total mathematical function; it does not model divergence or exceptions in a Haskell callback.

20.4 Closures and higher-order functions

Ordinary Haskell allows `map (+k) xs`; `SL.map (+k) xs` needs an explicit `Closure` when `k` is a symbolic unknown from the enclosing solver context:

```
shiftEach :: SInteger -> SList Integer -> SList Integer
shiftEach k = SL.map Closure { closureEnv = k
                             , closureFun = \env x -> x + env
                             }
```

SBV passes `k` through the recursion as `env`, so `closureFun` need not capture it. Literals and globally named logical symbols need no such environment. The companion proves length preservation for every offset and finite integer list.

21 A verified tautology checker

The checker in this section is a small verified decision procedure for propositional logic. It accepts a finite formula exactly when the formula is true under every assignment, and it can produce a falsifying assignment when the answer is no. Its design follows a classic development by Boyer and Moore (1979, Chapter 4), later formalized in Imandra (Passmore nd). The SBV development is `Documentation.SBV.Examples.TP.TautologyChecker`. We follow the algorithm and its main proof obligations; listings below are extracted from that module, with whitespace adjusted in the longer declarations. The companion imports it qualified as `Taut`.

21.1 Formulas, assignments, and evaluation

A formula is a constant, a variable identified by an integer, or a conditional:

```
data Formula = FTrue
             | FFalse
             | Var { fVar   :: Integer }
             | If  { ifCond :: Formula
                   , ifThen :: Formula
                   , ifElse :: Formula
                   }
```

Conditionals suffice to encode the familiar connectives. Writing p and q for formulas, negation is `If p FFalse FTrue`, conjunction is `If p q FFalse`, and implication is `If p q FTrue`. These are formulas of the object language; `If` constructs a formula rather than evaluating a Haskell conditional.

An assignment is a list of `Binding` values, each pairing a variable ID with a Boolean. `lookUp` uses the first matching binding and defaults to false if none is present. Evaluation is structural recursion:

```
eval :: SFormula -> SList Binding -> SBool
eval = smtFunction "eval"
     $ \f bs -> [sCase| f of
                  Var n    -> lookUp n bs
                  If c l r | eval c bs -> eval l bs
                           | True      -> eval r bs
                  FTrue    -> sTrue
                  FFalse   -> sFalse
                |]
```

This gives the specification independently of the checking algorithm. A tautology must evaluate to true for *every* assignment, not just the empty assignment. Defaults make evaluation total; they do not mean that a variable is fixed to false when asking whether a formula is a tautology. Every assignment to the finitely many variables of a formula can be represented by a finite binding list.

We will follow two examples: $p \Rightarrow p$, represented by `If (Var 0) (Var 0) FTrue`, and $p \Rightarrow q$, represented by `If (Var 0) (Var 1) FTrue`. The first is a tautology. The second is false when variable 0 is true and variable 1 is false.

21.2 Normalization and a decreasing measure

The checking traversal expects a normal form in which the condition of every `If` is a constant or a variable. A conditional can occur in either branch, but not as another conditional's condition. Normalization repeatedly distributes a nested condition:

$$\mathit{If}\ (\mathit{If}\ p\ q\ r)\ l\ r' \quad\longmapsto\quad \mathit{If}\ p\ (\mathit{If}\ q\ l\ r')\ (\mathit{If}\ r\ l\ r').$$

If p is true, both sides choose between l and r' using q; otherwise they use r. The rewrite therefore preserves meaning. For example, the condition of $(p \wedge q) \Rightarrow q$ is initially an `If`; this rewrite exposes p as the outer test. The complete definition supplies both the measure and its supporting lemmas:

```
normalize :: SFormula -> SFormula
normalize = smtFunctionWithMeasure "normalize"
  ( \f -> tuple (ifComplexity f, ifDepth f)
  , [ measureLemma ifDepthNonNeg
    , measureLemma ifComplexityPos
    , measureLemmaWith z3 ifComplexitySmaller
    , measureLemmaWith z3 normalizePreservesComplexity
    ]
  ) $ \f -> [sCase| f of
    If (If p q r) left right ->
      normalize (sIf p (sIf q left right) (sIf r left right))
    If c left right -> sIf c (normalize left) (normalize right)
    _ -> f
  |]
```

Termination is not structural: the first branch can produce a larger tree by duplicating l and r'. The supplied measure is a lexicographically ordered pair. Write C for `ifComplexity` and D for `ifDepth`. On constants and variables, $C = 1$ and $D = 0$; on conditionals,

$$C(\mathit{If}\ c\ l\ r) = C(c)\big(C(l) + C(r)\big), \qquad D(\mathit{If}\ c\ l\ r) = 1 + D(c).$$

The distributive rewrite preserves C and reduces D by one. Recursing into either branch of an ordinary conditional strictly reduces C, since every complexity is positive. Each call therefore decreases the pair even when tree size increases. Proved facts supplied through `measureLemma` justify these checks. Separate lemmas establish that normalization produces a normal formula and preserves its evaluation under every assignment.

21.3 Checking both branches consistently

The worker maintains bindings for variables already tested along the current path. Here is the complete traversal:

```
isTautology' :: SFormula -> SList Binding -> SBool
isTautology' = smtFunction "isTautology'" $ \f bs ->
  [sCase| f of
    -- Trivial cases
    FTrue          -> sTrue
    FFalse         -> sFalse

    -- Variable
    Var _          -> eval f bs

    -- Constant branches
    If FTrue  l _  -> isTautology' l bs
    If FFalse _ r  -> isTautology' r bs

    -- Branching on a variable
    If (Var n) l r
      -- We have already this variable, so evaluate based on the current choice
      | isAssigned n bs, eval (sVar n) bs -> isTautology' l bs
      | isAssigned n bs                   -> isTautology' r bs

      -- We haven't yet assigned this variable. Both branches should work out:
      | True             ->     isTautology' l (assumeTrue  n bs)
                           .&& isTautology' r (assumeFalse n bs)

    If _ _ _ -> sFalse  -- Contradicts isNormal assumption
  |]
```

Constants settle immediately. At an `If` whose condition is a known variable, the worker follows the branch fixed by its binding. For an unassigned variable it checks both branches, prepending a true binding for the left branch and a false binding for the right. Repeated occurrences must respect the same choice: they are not independent Boolean inputs. A bare variable is accepted only when its lookup is true; an unassigned bare variable is correctly rejected because choosing it false is a counterexample. The final case rejects a nested condition, which normalization rules out before the traversal starts:

```
isTautology :: SFormula -> SBool
isTautology f = isTautology' (normalize f) []
```

For $p \Rightarrow p$, the first test splits into two cases:

Path assumption	Remaining formula	Result
$p = \mathit{True}$	p	true by lookup
$p = \mathit{False}$	FTrue	true immediately

Both paths succeed. For $p \Rightarrow q$, the true branch reaches the unassigned variable q and fails; the false branch still succeeds. The conjunction of the two results is false. This is a search through consistent assignments, with branches pruned when a variable already has a value; it can still explore exponentially many cases.

The companion checks these examples and 70 formulas in total against an independent Haskell evaluator and all four assignments to variables 0 and 1. The collection includes nested conditions, so it also checks normalization and its truth preservation. These finite checks illustrate the algorithm; the following proofs establish the universal claims.

21.4 Soundness through a generalized invariant

The key lemma does not start with an empty environment. For every formula f, path bindings b, and further bindings a, it establishes

$$\mathit{isNormal}\ f\ \wedge\ \mathit{isTautology}'\ f\ b \quad\Longrightarrow\quad \mathit{eval}\ f\ (b +\!\!+ a).$$

The leading bindings b retain priority over the arbitrary completion a. This generalization is essential: recursive calls extend b, so a hypothesis only about the empty environment would not apply. It is the same design move used for the accumulator in reversal and Kadane's invariant.

The proof uses `sInduct` with `ifComplexity`. In the unassigned-variable case, checking has established both branch results, one with the variable true and one with it false. Split on that variable's value in the arbitrary completion. Apply the induction hypothesis to the corresponding branch and use the binding lemmas to show that its extended environment agrees with the original evaluation. When the variable is already assigned, only its selected branch matters. Positivity and branch-decrease lemmas discharge the measure obligations. This isolates three groups of reusable facts: lookup under extension, normalization, and correctness of the traversal under path assumptions.

The top-level soundness calculation assembles those facts:

```
soundness :: TP (Proof (Forall "f" Formula
                    -> Forall "bindings" [Binding] -> SBool))
soundness = do
  tie <- recallWith cvc5 tautologyImpliesEval
  nrt <- recall normalizeRespectsTruth
  nc  <- recall normalizeCorrect
  calc "soundness"
    (\(Forall f) (Forall bindings) -> isTautology f .=> eval f bindings) $
    \f bindings -> [isTautology f]
      |- eval f bindings
      ?? nrt `at` (Inst @"f" f, Inst @"bs" bindings)
      =: eval (normalize f) bindings
      ?? nc `at` Inst @"f" f
      ?? tie `at` (Inst @"f" (normalize f),
                   Inst @"a" bindings, Inst @"b" [])
      =: sTrue
      =: qed
```

First, `normalizeRespectsTruth` replaces evaluation of the original formula by evaluation of its normalization. Then `normalizeCorrect` supplies the normal-form premise, and `tautologyImpliesEval` is instantiated with empty path bindings and the arbitrary input assignment. The conclusion is true. The theorem covers every finite formula and every finite binding list; it is not limited to the formulas used in the concrete tests.

21.5 Completeness and a falsifying assignment

Rejection should also have a meaning. The function `falsify` normalizes its input and traverses it with bindings, like the checker, but seeks one failing path rather than requiring both paths to succeed. Its result records whether a counterexample was found and a list of bindings. A false constant returns the current bindings; an unassigned variable can be made false. At a new conditional test, the search tries the true branch and tries the false branch if the first attempt finds no counterexample.

For $p \Rightarrow q$, this produces $p = \mathit{True}, q = \mathit{False}$. Evaluating the *original* formula under that assignment gives false. The completeness theorem establishes this behavior for every rejected formula:

```
completeness :: TP (Proof (Forall "f" Formula -> SBool))
completeness = do
  ch  <- recall completenessHelper
  nrt <- recallWith z3 normalizeRespectsTruth

  lemma "completeness"
        (\(Forall f) -> sNot (isTautology f) .=> sNot (eval f (scex (falsify f))))
        [proofOf ch, proofOf nrt]
```

The helper theorem establishes falsification of the normalized formula; truth preservation transfers that result back to the original input. Combined with soundness, rejection has a concrete witness and acceptance holds under every assignment. The returned bindings are produced by our verified traversal, whereas the counterexamples in Section 19 are models supplied by the SMT solver when it refutes a proof obligation.

The companion runs both top-level proofs and their dependencies, with some obligations using CVC5. The proof structure follows the program structure: small lemmas establish lookup and normalization facts, a generalized induction handles the traversal, and short final arguments expose the public guarantees. This makes a verified algorithm understandable without reproducing every helper proof in the tutorial.

22 C generation from verified functions

22.1 A verified function across the translation boundary

A theorem about a symbolic function and a claim about an executable program meet at a translation boundary. To make that boundary concrete, we return to the first specification in the tutorial. SBV translates a symbolic function to C code that runs without a solver. For the saturating adder of Section 3:

```
generateAdder :: FilePath -> IO ()
generateAdder dir = compileToC (Just dir) "sat_add" $ do
  cgOverwriteFiles True
  cgGenerateDriver False
  x <- cgInput "x"
  y <- cgInput "y"
  cgReturn $ satAdd x y
```

This produces `sat_add.c` and a matching header. Because the C code is generated from the same `satAdd` that we proved correct, there is no manual rewrite in which the two could drift apart. For this adder, the generated header maps `SWord8` to `uint8_t`, and the arithmetic helpers implement 8-bit modular addition and subtraction using unsigned intermediates and masks.

22.2 Executable validation and trust assumptions

The companion checks the compiled artifact exhaustively: it compiles the generated adder with undefined-behavior sanitization and compares all 65,536 inputs with an independent oracle written directly in C. This checks the translation and compilation boundary for the generated executable; Section 24 explains the remaining trust assumptions.

Not all of SBV's vocabulary is executable. Quantifiers and uninterpreted sorts, for example, have no operational meaning, and the code generator rejects programs that use them rather than approximating them. Uninterpreted functions become calls to C functions that the user provides, which then join the trusted code.

23 The execution model and its proof obligations

Two questions connect the examples to their implementation: what does Haskell evaluate before the solver runs, and which definitions may the proof rely on? The representation of symbolic values explains the first; termination and the choice of a finite, total domain explain the second. These distinctions help predict both the cost of constructing a query and the scope of its answer.

Symbolic values are deferred graph nodes. Internally, a value of type `SBV a` is either a concrete constant or a deferred computation that, given the state of the current symbolic run, adds a node to a graph and returns its identifier:

```
newtype SBV a = SBV SVal
data SVal     = SVal Kind (Either CV (Cached SV))
newtype Cached a = Cached (State -> IO a)
```

The addition function `\x y -> x + y` can therefore have the pure type `SWord8 -> SWord8 -> SWord8` and still allocate graph nodes. Operations on constants are folded immediately, which is why `satAdd 250 10` evaluates to `255` without a solver. The run-time `Kind` records the logical sort, so simplifications can respect it: rewriting `x .== x` to true is valid for integers but not for floats.

Sharing comes in two forms. A generator that combines a term with itself, such as `iterate (\t -> t + t) x !! 30`, describes a term that is exponentially large as a tree but linear as a graph. SBV preserves this sharing in two ways. It memoizes each deferred computation,

using stable names in the style of observable sharing (Gill 2009), so that forcing the same Haskell value twice yields the same node. It also interns nodes, so that separately constructed applications of the same operator to the same arguments reuse a node. Neither mechanism identifies arbitrary mathematically equal expressions.

Nodes belong to a run. A graph node is meaningful only in the run that created it. Two values of type `SInteger` from different calls to `prove` cannot be combined, even though their types agree, and SBV checks for common mistakes of this kind. To carry a result from one run to another, extract a concrete value from a model and turn it back into a literal. Section 9 illustrates the same concrete-to-symbolic conversion with `literal v`, though its loop stays within one run. Constants and reusable function definitions can be shared across runs; session-specific symbolic unknowns cannot.

Queries synchronize dependencies. When a query creates a new term, SBV records the declarations it requires: sorts for datatypes (including those nested inside lists, tuples, or other datatypes), uninterpreted constants, and definitions of `smtFunctions` it calls. Before the term is used, any of these that the solver has not seen are sent. This is why a datatype can first appear inside a query. The companion exercises this case with two fresh values of a parameterized datatype created by `freshVar`.

23.1 Termination is a proof obligation

SMT functions denote total functions. An equation such as $b(n) = b(n) + 1$ has no integer-valued solution; admitting it as a definition would make the logical context inconsistent and allow unrelated claims to be proved. TP therefore checks termination of the symbolic functions used in a proof, separately from checking the theorem itself.

SBV tries automatic measures such as list length or datatype size. For recursion on a list's tail, length decreases at each call and is bounded below by zero, so the calls cannot continue indefinitely. The `Functions proven terminating` line reports these checks. If the automatic check fails, supply a measure with `smtFunctionWithMeasure`; the normalizer in Section 21 illustrates a lexicographic measure. An unknown solver result does not establish termination. Explicitly bypassing a check with `smtFunctionNoTermination` leaves a dependency reported as `Modulo` the function's termination. The companion checks that $b(n) = b(n) + 1$ is rejected.

Nested recursion can also require a postcondition to justify termination. Section 18 shows how `smtFunctionWithContract` checks such a postcondition together with the decreasing measure.

23.2 Finite lists and datatypes

Symbolic lists and recursive algebraic datatypes contain finite, total values: no undefined elements, partial tails, or infinite constructor trees. Sizes are unbounded, but each value is finite. For example, double reversal is the identity on finite lists, while for `xs = repeat 0`, `take 1 xs` returns `[0]` and `take 1 (reverse (reverse xs))` diverges. This domain makes structural induction and decreasing-size arguments available; total reasoning and its relationship to partial languages are studied by Danielsson et al. (2006). Arrays instead denote total mappings over possibly infinite index domains (Section 12); they do not introduce infinite recursive constructor structure. This does not rule out proving properties of streams:

Section 23.3 distinguishes productivity from termination and discusses representations for reasoning about stream observations.

23.3 Productivity and stream properties

A stream producer need not terminate: instead, each finite observation should be obtainable after finite work. This is *productivity*, a different obligation from reaching a final result. SBV provides `smtProductiveFunction` to request a constructor-guardedness check rather than a decreasing measure, and TP reports such checks as `Functions proven productive`. The examples `Documentation.SBV.Examples.TP.NatStream` and `MutualCorecursion` illustrate this interface.

The representation still matters. A guardedness check does not turn SMT sequences into infinite lists, nor supply a coinductive datatype model; the finite-domain restriction above continues to apply. For stream properties, one can instead represent a stream by an index function and prove pointwise claims, or define a terminating generator of finite prefixes and reason about those prefixes by induction. For example, the stream of integers starting at n has index function $s_n(k) = n+k$ for $k \geq 0$; its successor law $s_n(k+1) = s_n(k)+1$ is an ordinary universally quantified arithmetic property. Such proofs concern the chosen representation and observations. They do not by themselves establish productivity of a separate Haskell stream producer or extend finite-list theorems to infinite constructor structures.

24 Proof results, diagnostics, and trust

24.1 Failed checks and diagnostic steps

Before proving, run `testSatAdd` to check `satAddSpec` on random byte inputs with `sbvQuickCheck`. The companion checks this result before proving; passing random tests is not a proof.

Pass `"sat-add.smt2"` to this helper to inspect solver commands:

```
inspectSatAdd :: FilePath -> IO ThmResult
inspectSatAdd path = proveWith
  z3{verbose = True, transcript = Just path} satAddSpec
```

Here `verbose` prints abbreviated diagnostics; `transcript` writes SMT-LIB. Replay it outside GHCi with `z3 sat-add.smt2`.

The solver cannot start. Run `z3 --version` in the same shell and check that the executable is on `PATH`. Installing SBV does not install Z3.

The solver returns unknown or times out. No conclusion follows. Simplify the obligation and check whether its theories suit the solver; nonlinear arithmetic is a common obstacle. Use `z3Timeout5` from Section 8 to bound each solver call to 5,000 milliseconds. `SetTimeOut` does not limit the whole TP run, which may make many calls.

A counterexample is found. Evaluate implementation and specification on that input. The result of `wrongAdd 223 124` is 91. The specification evaluates `min 255 (223 + 124 :: Integer)` to 255, which agrees with the corrected `satAdd`. Check which side disagrees with the intended behavior.

A TP calculation stalls. Find the failed step and distinguish a counterexample from unknown. Check its assumptions and the `??` hints on that equality, instantiate hypotheses explicitly, or split the step with an intermediate expression. If the recursive call changes an accumulator, generalize the statement as in Sections 15 and 16. Section 19 shows concrete TP diagnostics and how to interpret the models of failed obligations.

Common pitfalls.

- *Wrong domain.* A property proved over `SInteger` says nothing about `SInt32`, and vice versa. Choose the type that matches the code you care about. List and recursive-datatype theorems cover finite, total values only (Section 23.2).
- *Specification repeats the implementation.* Write specifications in a different style or a wider domain (Section 3).
- *Haskell `if` or recursion on symbolic data.* Use `ite` and `sCase` for symbolic choices, and `smtFunction` for recursion over symbolic structure (Section 8).
- *Contradictory assumptions.* If `constrained` assumptions are unsatisfiable, everything is provable. A quick `sat` on the assumptions alone detects this.
- *Mixing runs.* Unknowns belong to the run that created them; move concrete values instead (Section 23).

What a proof rests on. A Q.E.D. from SBV depends on the correctness of SBV's translation to SMT-LIB, the solver, and, for TP, the generation of proof obligations. TP proofs also depend on any axioms the user introduces and on any functions whose termination checks were explicitly bypassed. SBV tracks these dependencies: a proof that rests on an axiom, or on an unfinished step marked with `sorry`, is reported as `Modulo` the offending lemmas rather than as `Proven`. TP does not replay solver certificates in an independent proof kernel. Functional correctness does not establish running-time or space bounds. For generated C, the C compiler and code generator join the trusted base: Section 22's exhaustive check covers the tested executable and input domain, rather than verifying either compiler in general.

24.2 Encodings and solver selection

Solver selection starts with the meaning of the formula. Integer addition and byte addition are not interchangeable encodings of the same operation unless the relationship is stated. For example, this bridge theorem relates modular byte addition to an integer expression:

```
byteAdditionBridge :: SWord8 -> SWord8 -> SBool
byteAdditionBridge x y =
  sFromIntegral (x+y) .== (a+b) `sMod` 256
  where a = sFromIntegral x :: SInteger
        b = sFromIntegral y :: SInteger
```

`prove byteAdditionBridge` checks the connection rather than assuming it. If an abstraction omits the modulus, it needs a no-overflow precondition. Similarly, replacing a finite list by an integer-indexed array introduces a total memory model; length and bounds must be carried separately.

After fixing semantics, compare formulations by inspecting the emitted problem and measuring the actual obligations. A finite Boolean circuit, quantified integer formula, and recursive sequence definition expose different work to a solver. No universal ranking of

backends follows. The companion checks the byte field theorem with both `proveWith z3 fieldRoundTrip` and `proveWith cvc5 fieldRoundTrip`; changing the backend does not change the intended statement, but support and effectiveness depend on the theories used. Optimization remains a Z3-specific SBV interface in this tutorial.

When a formulation stalls, first identify whether the difficulty is recursion, quantifier instantiation, nonlinear arithmetic, or the size of a finite encoding. Then try a justified reformulation or supply a lemma. Keep the model and its oracle unchanged while comparing attempts, preserve timeouts, and record solver versions and transcripts. A faster answer to a weakened property is not a successful optimization of the proof task.

24.3 Model validation and proof dependencies

A printed `Q.E.D.` is only part of the report. TP also records dependencies, including unproved steps and user axioms. The following deliberately checks the same true identity twice, once normally and once with `sorry` supplied:

```
auditProofs :: IO Bool
auditProofs = do
  clean <- runTP $ lemma "checkedIdentity"
    (\(Forall n :: Forall "n" Integer) -> n+n .== 2*n) []
  draft <- runTP $ lemma "draftIdentity"
    (\(Forall n :: Forall "n" Integer) -> n+n .== 2*n) [sorry]
  putStrLn $ showProofTree True draft
  print $ rootOfTrust draft
  pure $ case (rootOfTrust clean, rootOfTrust draft) of
    (RootOfTrust Nothing, RootOfTrust (Just _)) ->
      "Modulo" `isInfixOf` show draft
    _ -> False
```

The normal proof has `RootOfTrust Nothing`; the draft has a nonempty root of trust and is displayed as `Modulo: sorry`. The dependency tree shows the path from `draftIdentity` to `sorry`. This remains a draft even though its statement happens to be true. Removing the placeholder and rerunning the proof is what establishes the checked version.

`showProofTree True` compresses repeated uses of a dependency; use `False` to retain them. `rootOfTrust` identifies the outstanding trusted dependencies. `axiom` introduces a proposition without proving it, and bypassing a function's termination check creates another trust dependency. These are different from explicit premises of a conditional theorem, whose logical role is visible in the statement. Inspect both the theorem and its root of trust before reusing it.

For refutations, audit the witness as well. Re-evaluate concrete inputs with an independent interpreter or arithmetic calculation, check function defaults and array updates, and distinguish a local proof-step model from a program input. The validation driver does this for the examples in Section 19. For successful checks, dependencies explain what was assumed; they do not replace the solver and translation components of the trusted base described above. Keep the checked source, solver configuration, and validation record together so the claim remains reproducible.

25 Connections to program reasoning

The examples place SBV within a broader set of approaches to reasoning about functional programs. Three questions make the connections useful: how is the program represented, how is a proof decomposed, and what does a successful check establish?

25.1 Testing and symbolic evaluation

QuickCheck (Claessen and Hughes 2000) treats a property as an executable function and searches for counterexamples by sampling inputs. Symbolic reasoning instead represents unknown inputs and asks whether a violating assignment exists. For `satAddSpec`, the independent specification serves both purposes: Section 24.1 tests it before asking the solver to prove it. The practical connection is a shared specification and concrete feedback, while the evidence differs: successful tests cover the executed inputs, and an unsatisfiability result rules out a counterexample throughout the modeled domain.

Rosette (Torlak and Bodík 2014) develops solver-aided host languages using a symbolic virtual machine, demonstrated in Racket. Grisette (Lu and Bodík 2023) brings symbolic evaluation and state merging to Haskell as a library; Smten (Uhler and Dave 2013) translates high-level symbolic computations in a Haskell-derived language to SMT queries. These approaches address a common representation problem: execution with unknown data must preserve the possible outcomes of symbolic choices. In our examples, `ite` and `sCase` make those choices explicit, while ordinary Haskell computes the query generator. The what4 library (Galois, Inc. 2026) provides typed symbolic values and solver interfaces for building verification tools. SBV's interface combines symbolic programming, model extraction, and explicit proof calculations in one library.

25.2 Equational reasoning and proof guidance

Danvy (2022) teaches equational reasoning about recursive programs in Coq using fold–unfold lemmas. A recursive definition supplies equations for its constructor cases, and controlled rewriting makes their use visible in a proof. Our mirror calculation illustrates a related teaching choice: unfold the transformation, apply the hypotheses for both children, and then use commutativity. The displayed steps explain the argument even when automation could combine several of them. The checking mechanisms differ: our `=:` links generate SMT obligations, and hints provide facts for the following link.

The expression evaluator also connects to tutorials on programming-language semantics. Hutton (2023) uses a minimal arithmetic language to develop semantic descriptions and calculate an abstract machine. Our evaluator serves as the specification of a transformation: `mirrorCorrect` proves that changing syntax preserves meaning. Reusing `Expr` for search and proof lets the reader see the change from finding one term with a desired evaluation to establishing an equation for every finite term.

Generalization is the crucial step when the computation carries extra state. The reversal proof needs a statement about an arbitrary accumulator because a recursive call changes it; Kadane's proof needs an invariant describing the remaining suffix under arbitrary admissible accumulator values. In each case, the stronger statement makes the induction step possible. This is the reusable lesson of the calculations, independently of their concrete proof notation.

25.3 Refinements, invariants, and the scope of a proof

Liquid Haskell (Vazou et al. 2014) adds logical refinements to Haskell types. Refinement reflection (Vazou et al. 2018) makes function definitions available to the refinement logic and supports equational proofs written as Haskell programs, with pattern matching and recursion expressing case splits and induction. It therefore shares our interest in combining user-written proof structure with SMT reasoning. The interface differs: Liquid Haskell checks refinements of Haskell programs, while our SBV scripts construct symbolic models and execute TP checks. A `Proof` value is obtained by running those checks; ordinary GHC typechecking of a script does not establish its theorem.

Dafny (Leino 2010) provides a language and verifier in which programmers supply specifications and invariants and the verifier uses SMT to discharge the resulting obligations. The accumulator invariant in our Kadane development plays a comparable explanatory role: it describes what remains true as the state changes, here across recursive calls rather than an imperative loop.

For readers moving between these approaches, it is useful to carry over the specification and the mathematical argument before translating the notation. Check which numeric domains, termination conditions, and notions of equality the target system uses. Our finite-data scope (Section 23.2) and monomorphic theorem instances (Section 20.1) are part of the claim being proved, not incidental details of the proof script.

26 Suggested projects

26.1 Project: from recursive invariants to state transitions

Transfer the invariant-discovery method from Kadane's algorithm to a program represented by successive states. Begin with integer state $(x, y) = (1, 1)$ and the simultaneous update

$$(x, y) \longmapsto (x + 1, y + x).$$

The safety property is $y \geq 1$ after any number of steps. Work through the following stages; this is an open-ended project rather than an additional worked solution in the companion.

(1) Encode the initial state, transition relation, and unsafe states. Search for an execution reaching $y < 1$ within a chosen bound. Explain why finding no such execution establishes safety only up to that bound.
(2) Try to prove that $y \geq 1$ holds initially and is preserved by every transition from a state satisfying it. Inspect a counterexample to preservation and decide whether it is reachable from $(1, 1)$.
(3) Strengthen the invariant so that both initialization and preservation can be proved. Explain why the strengthened invariant establishes safety at every finite depth, and compare this strengthening with generalizing the accumulator hypothesis in Section 16.
(4) As an extension, express a bounded loop performing the updates using the weakest-precondition interface. Separate its safety assertion from the extra argument needed to establish termination.

Starting hints. Use mathematical integers first, so overflow does not change the question. The next value of y depends on x; a lower bound on x may therefore be needed even though the requested safety property mentions only y. For example, $(-1, 1) \longmapsto (0, 0)$ refutes preservation of $y \geq 1$ alone, but is not an execution starting from the specified initial state. A

counterexample to inductiveness is an invitation to inspect the invariant, not automatically a counterexample to safety.

For starting code, see these modules under `Documentation.SBV.Examples`:

- `ProofTools.BMC` demonstrates bounded reachability with a state datatype and a transition relation.
- `ProofTools.Strengthen` contains this update and examples of invariant strengthening.
- `WeakestPreconditions.Basics` introduces the imperative-program interface.

Adapt the BMC example's state and goal to this system rather than expecting its original transition relation to match. Keep the concrete simulator independent of the symbolic encoding and compare short traces before attempting the universal argument.

26.2 Project: mechanizing Bird's maximum-segment-sum derivation

Revisit maximum segment sum by following the algebraic derivation of Bird (1989). The goal is to calculate an efficient implementation from a specification, with TP checking the intervening transformations. Start with finite lists of mathematical integers and the specification `maximum . map sum . segs`, where `segs` enumerates every contiguous segment, including the empty segment. This ensures a result of 0 for empty or all-negative inputs. Use the independent segment oracle from Section 16 for initial concrete checks.

(1) Express segment enumeration using prefixes and suffixes, and formulate the transformations as equalities on an arbitrary symbolic input list. Implement any missing symbolic prefix, suffix, or scan operations, with their termination obligations.

(2) Prove the laws needed to reach a scan: addition distributes over binary maximum, Horner's rule combines the suffix sums, and the scan lemma shares work across prefixes. Begin with the integer instances needed here; generalizing the laws to arbitrary operations is an optional extension.

(3) Fuse the scan and the final maximum into a paired left fold. Identify its components with `msf` and `meh`: in the pair `(msf, meh)`, the first records the best sum seen so far and the second the best sum ending at the current position; `kadaneHelper` takes these arguments in the opposite order. Prove that the fold's first component agrees with `kadane`. Assemble the intermediate theorems into an end-to-end proof from segment enumeration to the paired fold.

Starting hints. Study `foldrMapFusion` and `foldrFusion` in `Documentation.SBV.Examples.TP.Lists` as examples of how to state and discharge fusion side conditions; Bird's left-fold calculation will need the corresponding laws for its chosen operations. State function equalities pointwise, and instantiate helper lemmas explicitly, as in the bookkeeping proof in Section 20. Keep a record of which steps need a supplied lemma and which the solver closes from definitions. Compare the resulting proof effort with the accumulator invariant of Section 16. This is an advanced, open-ended project: the companion does not contain a mechanization of Bird's derivation, and its compact paper calculation should not be taken as an estimate of the TP work.

27 Conclusion

Solver-aided reasoning combines executable feedback with explicit mathematical structure. The examples develop a repeatable method: choose the symbolic domain, write an independent specification, and use counterexamples to check both the code and its intended behavior. When automation stalls, expose intermediate equalities and generalize the induction hypothesis to cover changing accumulators. Check termination separately, and read each theorem within its modeled domain. The resulting progression makes both success and failure informative: a counterexample helps refine a specification, and an unresolved proof obligation can expose the need for a stronger invariant or a more explicit argument. SBV supplies an executable setting for learning these decisions alongside the programs they justify.

For further practice, three SBV example modules extend these ideas. `ConstFold` proves that constant folding preserves expression meaning; `InsertionSort` combines ordering and permutation properties; and `VM` verifies compilation to a stack machine. All three are under `Documentation.SBV.Examples.TP`.

The neighboring `AES` example illustrates what TP enables beyond one-shot automation. SBV's existing implementation had a round-trip property—decrypting an encrypted plaintext returns the original—that had remained unproved because a single end-to-end SMT query was too demanding. With TP, the same implementation now has a compositional proof for every AES-128 plaintext and key, using Z3 and CVC5. Explicit proof structure makes this previously impractical check possible.

Acknowledgments

We thank Joel Burget, Iavor Diatchki, Brian Huffman, John Matthews, Brian Schroeder, and Jeffrey Young for their contributions to SBV's design and development, and the wider SBV community for bug reports, feedback, and examples.

References

Roberto Baldoni, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. *Comput. Surveys* 51, 3, Article 50 (2018), 39 pages. https://doi.org/10.1145/3182657

Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 13243)*. Springer, Cham, Switzerland, 415–442. https://doi.org/10.1007/978-3-030-99524-9_24

Clark Barrett, Pascal Fontaine, Silvio Ranise, and Cesare Tinelli. 2025b. SMT-LIB: Fixed-Size Bit-Vectors. https://smt-lib.org/theories-FixedSizeBitVectors.shtml Theory FixedSizeBitVectors, SMT-LIB 2.7, updated 9 September 2025.

Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2025a. *The SMT-LIB Standard: Version 2.7*. Technical Report. Department of Computer Science, The University of Iowa. https://smt-lib.org/papers/smt-lib-reference-v2.7-r2025-07-07.pdf Release of 7 July 2025.

Jon Bentley. 1984. Programming Pearls. *Commun. ACM* 27, 9 (1984), 865–873. https://doi.org/10.1145/358234.381162

Richard S. Bird. 1989. Algebraic Identities for Program Calculation. *Comput. J.* 32, 2 (1989), 122–126. https://doi.org/10.1093/comjnl/32.2.122

Jasmin Blanchette. 2026. *Hammering Away: A User's Guide to Sledgehammer for Isabelle/HOL*. https://isabelle.in.tum.de/doc/sledgehammer.pdf Version of 24 October 2026; with contributions from Martin Desharnais-Schäfer, Lawrence C. Paulson, and Lukas Bartl.

Joshua Bloch. 2006. Extra, Extra—Read All About It: Nearly All Binary Searches and Mergesorts are Broken. Google Research Blog, 2 June 2006. https://research.google/blog/extra-extra-read-all-about-it-nearly-all-binary-searches-and-mergesorts-are-broken/

Robert S. Boyer and J Strother Moore. 1979. *A Computational Logic*. Academic Press, New York. https://www.cs.utexas.edu/~boyer/acl.pdf

Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming*. ACM, New York, NY, USA, 268–279. https://doi.org/10.1145/351240.351266

Nils Anders Danielsson, John Hughes, Patrik Jansson, and Jeremy Gibbons. 2006. Fast and Loose Reasoning is Morally Correct. In *Conference Record of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, New York, NY, USA, 206–217. https://doi.org/10.1145/1111037.1111056

Olivier Danvy. 2022. Fold–unfold lemmas for reasoning about recursive programs using the Coq proof assistant. *Journal of Functional Programming* 32, Article e13 (2022), 27 pages. https://doi.org/10.1017/S0956796822000107

Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction—CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

Levent Erkök. 2026. SBV: SMT Based Verification in Haskell. Haskell library; tutorial checked with the 14.10 source tree. https://github.com/LeventErkok/sbv

Levent Erkök and Nikolaj Bjørner. 2021. Incorrect Real Optimization Results. Z3 issue 5314 and discussion. https://github.com/Z3Prover/z3/issues/5314

Galois, Inc. 2026. what4: Solver-agnostic symbolic values support for issuing queries. Hackage package `what4`, version 1.8. https://hackage.haskell.org/package/what4-1.8

Andy Gill. 2009. Type-Safe Observable Sharing in Haskell. In *Proceedings of the Second ACM SIGPLAN Symposium on Haskell*. ACM, New York, NY, USA, 117–128. https://doi.org/10.1145/1596638.1596653

Graham Hutton. 2023. Programming language semantics: It's easy as 1,2,3. *Journal of Functional Programming* 33, Article e9 (2023), 26 pages. https://doi.org/10.1017/S0956796823000072

Matt Kaufmann, Panagiotis Manolios, and J Strother Moore. 2000. *Computer-Aided Reasoning: An Approach*. Kluwer Academic Publishers. https://www.cs.utexas.edu/~moore/publications/acl2-books/car/index.html

James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (1976), 385–394. https://doi.org/10.1145/360248.360252

K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (Lecture Notes in Computer Science, Vol. 6355)*. Springer, Berlin, Heidelberg, 348–370. https://doi.org/10.1007/978-3-642-17511-4_20

Sirui Lu and Rastislav Bodík. 2023. Grisette: Symbolic Compilation as a Functional Programming Library. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 16 (2023), 33 pages. https://doi.org/10.1145/3571209

Microsoft. n.d.a. Online Z3 Guide: Arithmetic. https://microsoft.github.io/z3guide/docs/theories/Arithmetic/ Accessed 4 November 2026.

Microsoft. n.d.b. Online Z3 Guide: Arithmetical Optimization. https://microsoft.github.io/z3guide/docs/optimization/arithmeticaloptimization/ Accessed 4 November 2026.

Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. 2002. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Lecture Notes in Computer Science, Vol. 2283. Springer. https://doi.org/10.1007/3-540-45949-9

Ulf Norell. 2007. *Towards a Practical Programming Language Based on Dependent Type Theory*. Ph. D. Dissertation. Chalmers University of Technology. https://www.cse.chalmers.se/~ulfn/papers/thesis.pdf

Sam Owre, John M. Rushby, and Natarajan Shankar. 1992. PVS: A Prototype Verification System. In *11th International Conference on Automated Deduction (CADE) (Lecture Notes in Artificial Intelligence, Vol. 607)*. Springer, 748–752. https://www.csl.sri.com/papers/cade92-pvs/

Grant Passmore. n.d.. Tautology Checker Example for ImandraX (`tautology.iml`). Imandra examples repository. https://github.com/imandra-ai/imandrax-examples/blob/main/src/tautology.iml Descriptive title; source file accessed 4 November 2026.

Konrad Slind and Michael Norrish. 2008. A Brief Overview of HOL4. In *Theorem Proving in Higher Order Logics (Lecture Notes in Computer Science, Vol. 5170)*. Springer, 28–32. https://doi.org/10.1007/978-3-540-71067-7_6

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Vijay Saraswat, and Sanjit Seshia. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM Press, 404–415. https://people.csail.mit.edu/asolar/papers/asplos06-final.pdf

The Rocq Development Team. n.d.. *The Rocq Prover Reference Manual*. https://rocq-prover.org/doc/V9.2.0/refman/index.html Version 9.2.0; accessed 9 November 2026.

Cesare Tinelli. 2024. SMT-LIB: Arrays with Extensionality. https://smt-lib.org/theories-ArraysEx.shtml Theory ArraysEx, SMT-LIB 2.7, release of 21 July 2024.

Cesare Tinelli, Clark Barrett, and Pascal Fontaine. 2025. SMT-LIB: Unicode Strings. https://smt-lib.org/theories-UnicodeStrings.shtml Theory Strings, SMT-LIB 2.7, updated 3 December 2025.

Cesare Tinelli and Martin Brain. 2024. SMT-LIB: Floating Point. https://smt-lib.org/theories-FloatingPoint.shtml Theory FloatingPoint, SMT-LIB 2.7, release of 21 July 2024.

Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY, USA, 530–541. https://doi.org/10.1145/2594291.2594340

Richard Uhler and Nirav Dave. 2013. Smten: Automatic Translation of High-Level Symbolic Computations into SMT Queries. In *Computer Aided Verification*. Springer, Berlin, Heidelberg, 678–683. https://doi.org/10.1007/978-3-642-39799-8_45

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement Types for Haskell. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming*. ACM, New York, NY, USA, 269–282. https://doi.org/10.1145/2628136.2628161

Niki Vazou, Anish Tondwalkar, Vikraman Choudhury, Ryan G. Scott, Ryan R. Newton, Philip Wadler, and Ranjit Jhala. 2018. Refinement Reflection: Complete Verification with SMT. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 53 (2018), 31 pages. https://doi.org/10.1145/3158141

Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. 2023. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. https://doi.org/10.52202/075280-0944